



UNIVERSITÀ DEGLI STUDI DI MILANO - BICOCCA
**Dipartimento di Informatica, Sistemistica e Comuni-
cazione**
Corso di Laurea in Informatica

Esplorazione e Valutazione di una piattaforma FAAS

Relatore: Prof. Michele Ciavotta

Tesi di Laurea di:
Luca Basanisi
Matricola 904947

Anno Accademico 2025-2026

Indice

1	Obiettivo della tesi	4
1.1	Piattaforme FAAS e nanofaas	4
1.2	Obiettivo del lavoro	7
2	Introduzione degli strumenti utilizzati	8
2.1	Multipass	8
2.2	Container	9
2.3	Docker	10
2.4	Kubernetes	11
2.5	Il ruolo di Git	13
3	Nanofaas	15
3.1	Panoramica dell'architettura	16
3.2	Il Control Plane	17
3.2.1	API Gateway	17
3.3	Architettura modulare	18
3.3.1	Meccanismo di caricamento	18
3.3.2	Moduli principali	18
3.4	Modalità di esecuzione	19
3.4.1	LOCAL	19
3.4.2	POOL	19
3.4.3	DEPLOYMENT	19
3.5	Flusso di esecuzione delle invocazioni	20
3.5.1	Invocazione sincrona	20
3.5.2	Invocazione asincrona	20
3.5.3	Gestione dei tentativi e timeout	20
3.6	I runtime delle funzioni	21
3.6.1	Java Runtime	21
3.6.2	Python Runtime	21
3.6.3	SDK per sviluppatori	21
3.7	Osservabilità	21
4	Metodologia di Esplorazione e Valutazione	23
4.1	Approccio iniziale alla piattaforma	23
4.2	Documentazione rapida dei comandi	23
4.3	Fine dell'esplorazione informale	24
4.4	Esplorazione Automatizzata	24
4.5	Vantaggi del nuovo approccio	26

5	Risultati dell'analisi	27
5.1	Premesse sui risultati	27
5.1.1	Mancanza di commenti al codice	27
5.2	Esecuzione dei test	27
5.2.1	Avvio della Piattaforma	27
5.2.2	Test e2e-k8s-vm	29
5.2.3	Cluster Kubernetes mancante	30
5.3	Modalità Sviluppo in Locale	32
5.3.1	Le funzioni in nanofaas	33
5.4	Problemi di documentazione	35
5.4.1	Utilizzo di nanofaas-cli	35
5.4.2	Scaffolding	35
6	Automatizzare l'esplorazione	38
6.1	La struttura degli script	38
6.1.1	Justfile	38
6.1.2	Setup del sistema	39
6.2	Script	40
6.2.1	Test API	40
6.2.2	Multi Stage in Go	40
6.2.3	Test Comunicazione Asincrona	41
6.2.4	Idempotenza	43
6.2.5	Dipendenze Mancanti	44
6.2.6	CallBack Loop	46
7	Analisi degli SDK	47
7.1	Da un'API Comune alla frammentazione	47
7.2	Gestione dei dati e dei metadati	48
7.3	Tracciamento delle richieste e contesto	49
7.3.1	L'approccio Esplicito di Go	49
7.3.2	L'approccio Implicito di Java	50
7.3.3	L'evoluzione Asincrona di Python	51
7.4	Inizializzazione e stati di salute apparente	51
7.4.1	La gestione del caricamento nei diversi SDK	51
7.4.2	Il problema della salute apparente	52
7.5	Conclusioni	54
8	Conclusione	55
8.1	Valutazione della Maturità di nanofaas	56

Elenco delle figure

1.1	Architettura di un servizio FAAS	6
2.1	VCS-Generico	13
2.2	Git	14
4.1	Ciclo di vita completo di una sessione di test esplorativo assistito da strumenti	25
5.1	script <code>e2e-k8s-vm.sh</code>	29
5.2	Architettura nanofaas: separazione dei processi in modalità <code>LOCAL</code>	34
5.3	Relazione e comportamento delle modalità di esecuzione (<code>executionMode</code>) in nanofaas	37

Capitolo 1

Obiettivo della tesi

1.1 Piattaforme FAAS e nanofaas

Ambienti Cloud Native La gestione delle risorse computazionali rappresenta da sempre una delle principali sfide nello sviluppo e nell'esercizio delle applicazioni web. Nelle prime fasi di diffusione di Internet, le aziende erano generalmente responsabili dell'acquisto, della configurazione e della manutenzione della propria infrastruttura hardware. Tale approccio richiedeva un significativo investimento iniziale e rendeva complesso il dimensionamento delle risorse in funzione del carico di lavoro previsto.

La capacità dell'infrastruttura doveva infatti essere pianificata anticipatamente, cercando di stimare il numero di utenti e il volume di traffico che l'applicazione avrebbe dovuto gestire.

Una **sottostima** comportava degrado delle prestazioni o interruzioni del servizio, mentre una **sovrastima** comportava costi elevati dovuti a risorse inutilizzate.

Con l'evoluzione delle tecnologie informatiche sono stati introdotti modelli di erogazione delle risorse basati sul cloud computing, nei quali provider specializzati mettono a disposizione infrastrutture accessibili attraverso Internet. Questo modello consente alle aziende di acquisire risorse computazionali in modo dinamico, riducendo gli investimenti iniziali e demandando al provider gran parte della gestione dell'infrastruttura fisica.

Successivamente si è affermato il concetto di *cloud native*. Tale termine non identifica semplicemente applicazioni eseguite nel cloud, ma un insieme di principi architetturali e pratiche di sviluppo che consentono di realizzare sistemi scalabili, resilienti e facilmente distribuibili. Le applicazioni cloud native sono infatti progettate per sfruttare caratteristiche quali automazione, orchestrazione, elasticità e distribuzione continua, indipendentemente dall'ambiente di esecuzione.

Per questo motivo, un'applicazione può essere considerata cloud native anche quando viene eseguita al di fuori di un'infrastruttura cloud tradizionale,

ad esempio su dispositivi edge o in un data center privato, purché mantenga le caratteristiche architetturali proprie di questo paradigma. [4]

Function As a Service Un ambiente Function as a Service (FAAS) fornisce un servizio sotto forma di funzione che viene eseguita quando si verifica un determinato evento. Lo sviluppatore deve solamente implementare la logica della funzione, mentre la piattaforma si occupa automaticamente dell'avvio, dell'esecuzione e della gestione delle risorse necessarie. [34]

Le funzioni in un ambiente FAAS sono **effimere**, cioè vengono create per eseguire un unico compito, infatti non sono progettate né devono essere pensate per mantenere uno stato persistente tra un'esecuzione e la successiva.

Un'altra caratteristica distintiva del paradigma FAAS riguarda il modello di costo. A differenza di altri paradigmi, come PaaS o IaaS, in cui si paga per le risorse allocate indipendentemente dal loro utilizzo, nel FAAS il costo viene addebitato solo quando una funzione viene effettivamente invocata ed eseguita. Nelle applicazioni dove certe operazioni sono occasionali conviene adottare il paradigma FAAS perché si riducono i costi quando il servizio non viene utilizzato. [13], [18]

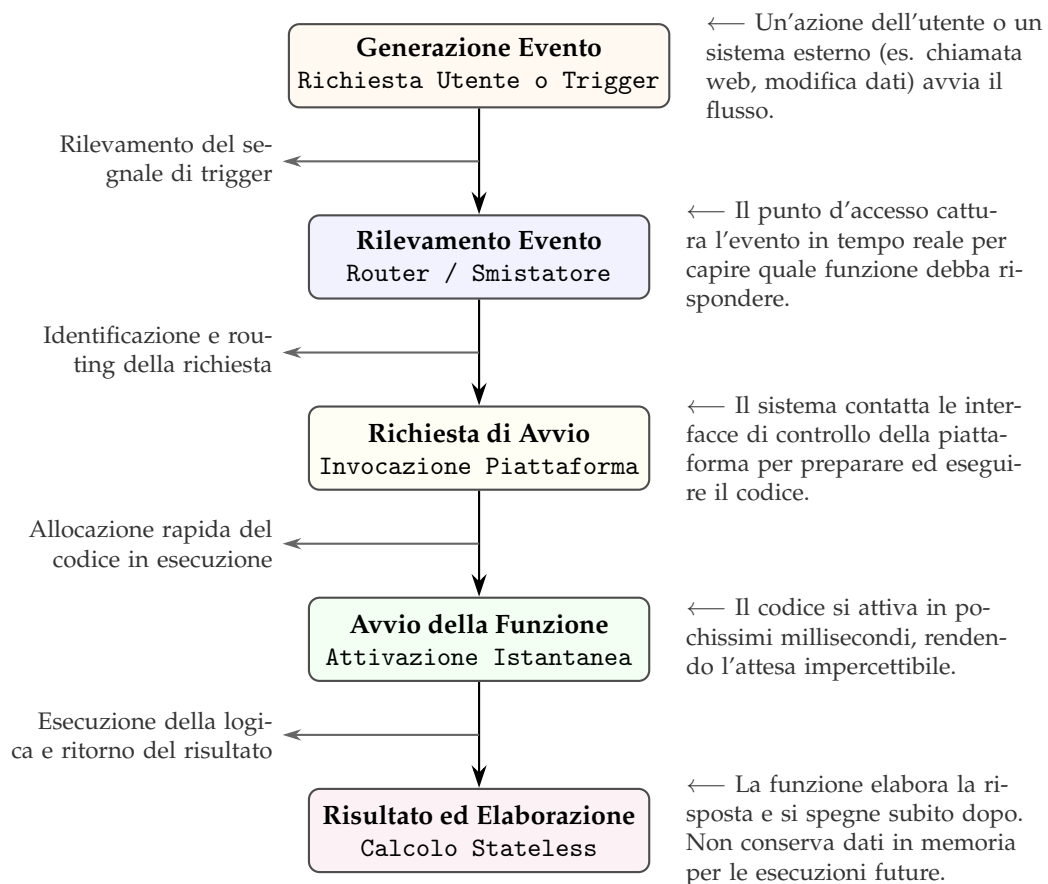


Figura 1.1: Architettura di un servizio FAAS

Come funziona FAAS? I vantaggi sono:

- le ridotte dimensioni del codice da mantenere
- le dimensioni del codice lo rendono scalabile
- la predisposizione al modello a microservizi
- la possibilità di avere dei team dedicati e di bilanciare meglio il lavoro (devops, cicd)

Svantaggi del FAAS L'adozione dei container al posto delle macchine virtuali tradizionali ha ridotto drasticamente i tempi di avvio di un servizio. Tuttavia, questo isolamento spesso non è ancora sufficientemente rapido quando l'obiettivo è eseguire una funzione in pochi millisecondi.

All'avvio di una nuova istanza si presenta il problema del **cold start**: il tempo richiesto per fare il download dell'immagine (se non presente in cache), istanziare il container, inizializzare il runtime e caricare il codice introduce una latenza iniziale spesso inaccettabile per applicazioni in tempo reale. [3]

Questo fenomeno crea un forte bias nella scelta dello stack tecnologico:

- Linguaggi compilati nativamente con tempi di avvio quasi istantanei, come Go, Rust o C, sono preferiti in ambienti FAAS.
- Linguaggi che richiedono l'avvio di macchine virtuali o runtime interpretati, come Java, sono meno graditi.

Stateless e complessità Le funzioni FAAS sono per definizione stateless: non conservano alcuna memoria tra un'invocazione e l'altra. Se da un lato questo facilita la scalabilità orizzontale, dall'altro sposta la complessità sul piano architetturale.

Per mantenere lo stato dell'applicazione è necessario delegare la persistenza a servizi esterni (per esempio Redis). Questo può aumentare la latenza e la complessità dell'applicazione.

1.2 Obiettivo del lavoro

L'obiettivo di questa tesi è analizzare e valutare la piattaforma nanofaas. [22] Nanofaas, come suggerisce il nome, è una piattaforma FAAS progettata secondo una filosofia minimale: anziché privilegiare la completezza funzionale, mira a offrire una soluzione leggera e ad alte prestazioni.

Essendo un progetto presente su GitHub, un ulteriore obiettivo del lavoro consiste nel familiarizzare con le dinamiche dello sviluppo open source, attraverso l'analisi del codice sorgente e l'eventuale segnalazione di problemi.

La valutazione svolta nel corso della tesi non è finalizzata a misurare quantitativamente le prestazioni della piattaforma. L'obiettivo è invece determinarne il grado di maturità tecnologica, verificando se le funzionalità e le caratteristiche dichiarate nella documentazione ufficiale siano effettivamente implementate e supportate dal sistema.

Capitolo 2

Introduzione degli strumenti utilizzati

2.1 Multipass

La VM Multipass è utilizzata per eseguire la piattaforma nanofaas in un ambiente di esecuzione controllato.

Multipass possiede un'integrazione nativa con cloud-init. Questo strumento permette di configurare automaticamente una VM appena viene avviata. [14], [15]

La configurazione avviene attraverso un file YAML, per esempio:

```
hostname: mia-vm
users:
  - name: luca
    sudo: ALL=(ALL) NOPASSWD:ALL
    ssh_authorized_keys:
      - ssh-rsa AAAAB3... tua-chiave

package_update: true
packages:
  - curl
  - git
  - docker.io
```

Immagini ISO Multipass inoltre è specializzato nelle distribuzioni Ubuntu, infatti usa delle immagini ottimizzate, chiamate Ubuntu Cloud Images. [14] Queste immagini:

- contengono solo il sistema di base, senza ambiente grafico
- includono già cloud-init e SSH
- sono ottimizzate per il cloud

SSH Quando si avvia una macchina virtuale l'unico modo per accedervi è tramite una chiave pubblica. Avendo la chiave già installata dentro la VM, grazie a cloud-init, è possibile accedervi tramite SSH.

La piattaforma nanofaas sfrutta tutte queste funzionalità di Multipass per creare un ambiente pulito in maniera veloce.

Inoltre, la possibilità di configurare un file YAML si sposa perfettamente con la filosofia CI/CD.

2.2 Container

Un container è un processo isolato del sistema operativo che contiene tutte le dipendenze necessarie per eseguire un altro software.

Il loro utilizzo principale è quello di sostituire le macchine virtuali, che concettualmente fanno la stessa cosa, ma sono estremamente esigenti in termini di risorse hardware, soprattutto per l'ambito cloud native.

Infatti il container non si appropria in maniera esclusiva dell'hardware della macchina, ma piuttosto lascia la gestione dell'hardware al kernel del sistema operativo HOST.

Un container quindi non può essere completamente isolato, in quanto di fatto è un processo del sistema operativo HOST e compete per le risorse hardware assieme agli altri processi.

Processi Un processo è un'istanza di un programma in esecuzione.

Il processo è sempre creato da un altro processo padre. Nel caso del kernel linux il PID 1 è il processo "init".

Quando avviamo un container, il suo processo padre è il container runtime. Il container runtime varia in base alla tecnologia che stiamo usando, ma in tutti i casi il suo compito principale è gestire il ciclo di vita del container

Runtime ad alto livello Il container runtime ad alto livello gestisce in maniera generale i suoi processi, senza interagire direttamente con il kernel. Coordina solamente i vari container in esecuzione e invia ordini al container runtime a basso livello. Il container runtime non serve unicamente a gestire internamente i container, ma ha anche lo scopo di esporre API pubbliche per far sì che altri servizi (come Docker o Kubernetes) possano a loro volta inviare comandi.

Runtime a basso livello Il container runtime a basso livello invece prende ordini dal container runtime ad alto livello e interagisce direttamente con il kernel del sistema operativo HOST.

Uno di questi, runc, fornisce le indicazioni al kernel su come creare il processo attraverso delle "syscall". Ma le caratteristiche che deve avere un container sono leggermente diverse rispetto a un processo normale. [27]

Componenti di un container Un container si differenzia da un processo standard grazie a:

- **Cgroups**
- **Namespaces**

Cgroups Questo componente impedisce che un processo (un container) consumi tutta la RAM o la CPU del sistema operativo HOST. Serve a limitare le risorse hardware a cui un processo può accedere. [16]

Namespaces È il componente del kernel Linux che si occupa di isolare i processi dal resto del sistema operativo. Serve a fornire un isolamento parziale al container, cercando di imitare le macchine virtuali. Il processo crede di avere PID 1 (come "init"), vede solo la sua interfaccia di rete virtuale (eth0) e vede solo una minuscola parte del filesystem. [37]

2.3 Docker

Docker nasce nel 2013. Il suo obiettivo è quello di rendere la gestione dei container più accessibile, grazie a comandi più intuitivi e interfacce più semplici. [17] La filosofia di Docker impone di isolare una singola applicazione, inserendola all'interno di un container.

Le innovazioni di Docker

- **Layer:** Le immagini di un container sono sovrapposte.
- **Dockerfile:** È forse la più importante innovazione di Docker, infatti permette di creare un file di configurazione che definisce istruzioni per costruire immagini.

Build e Deployment delle funzioni In nanofaas ogni funzione viene confezionata in un'immagine Docker a partire da un Dockerfile, un file che descrive passo dopo passo come costruire l'ambiente di esecuzione.

Una volta costruita, l'immagine viene pubblicata su un container registry con l'operazione di push, rendendola disponibile per l'esecuzione in qualsiasi ambiente.

L'esecuzione dell'applicazione avviene creando un **container** a partire dall'immagine. I layer read-only che compongono l'immagine vengono impilati tramite un union filesystem (overlay2) e Docker aggiunge sopra di essi un layer scrivibile in cui vengono memorizzate le modifiche effettuate durante l'esecuzione. Quando il container viene rimosso, questo layer scrivibile viene eliminato, mentre l'immagine originale rimane intatta e riutilizzabile. [17]

2.4 Kubernetes

Nel 2014, Kubernetes ha fatto il suo debutto come versione open source.

“Kubernetes è un sistema open source per eseguire il Deployment, scalare e gestire applicazioni containerizzate ovunque.”

Docker e Kubernetes Operano a livelli di astrazione differenti. Docker è un runtime per container: fornisce gli strumenti per costruire immagini, pubblicarle su un registry, avviare e arrestare container su una singola macchina e connetterli tra loro tramite reti virtuali. La sua unità di gestione è il singolo container.

Kubernetes, invece, è un orchestratore di container distribuiti: non esegue direttamente i container, per questo si appoggia a un container runtime come containerd o lo stesso Docker, ma gestisce un cluster di macchine come un'unica risorsa computazionale. Il suo compito è mantenere lo stato desiderato dell'applicazione in modo dichiarativo. L'utente specifica *cosa* vuole (numero di repliche, immagine da usare, risorse richieste) attraverso manifesti YAML e Kubernetes si occupa di riconciliare lo stato reale con quello desiderato.

Architettura di Kubernetes L'unità fondamentale di esecuzione è il **Pod**. Un Pod non è una macchina virtuale né un singolo container, ma un insieme di uno o più container che condividono alcune risorse del sistema operativo. In particolare, tutti i container di uno stesso Pod condividono il medesimo network namespace e quindi lo stesso indirizzo IP e lo stesso stack di rete. Questo permette ai processi nei diversi container di comunicare direttamente via localhost, usando semplicemente porte differenti. A seconda della configurazione, i container possono anche condividere altri namespace, come quelli per i volumi di archiviazione o per la comunicazione IPC.

Per la gestione delle risorse, Kubernetes delega ai **cgroups** del kernel Linux l'applicazione di vincoli su CPU e memoria, definiti nel manifesto YAML tramite i campi `requests` e `limits`.

Docker e Kubernetes In un contesto Kubernetes il flusso di build, push e deploy viene automatizzato: l'immagine, già costruita e pubblicata sul registry, viene scaricata automaticamente dai nodi del cluster Kubernetes nel momento in cui viene creato un nuovo Pod. In questo modo l'applicazione può essere distribuita in maniera riproducibile e indipendente dall'ambiente in cui verrà eseguita. [36] [35]

Deployment e Service Per orchestrare i Pod, Kubernetes mette a disposizione astrazioni di livello superiore. Un **Deployment** gestisce un insieme di repliche identiche di Pod, garantendo che il numero desiderato sia sempre in esecuzione.

Inoltre ad ogni Pod viene assegnato un indirizzo IP effimero, ma dato che i

Pod stessi sono effimeri, Kubernetes fornisce un'astrazione di rete stabile attraverso i **Service**. Un Service di tipo `ClusterIP` espone i Pod tramite un IP virtuale fisso e un nome DNS interno al cluster, bilanciando automaticamente il traffico tra tutte le repliche attive.

Per verificare che un container sia effettivamente pronto a ricevere richieste, si definisce una **readiness probe**, cioè un health check HTTP su un endpoint come `/health`, con un ritardo iniziale e un intervallo di polling configurabili. Solo i Pod che superano la readiness probe vengono aggiunti al bilanciamento del Service. [31]

Infrastructure Readiness e Application Readiness Quando un Pod viene avviato o riavviato, il sistema attraversa due fasi distinte che, pur sovrapponendosi temporalmente, sono concettualmente separate.

- L'**infrastructure readiness** è la condizione in cui il container runtime ha completato il proprio lavoro: il container è stato schedulato, i layer del filesystem sono stati montati, le interfacce di rete virtuali sono state create e l'indirizzo IP del Pod è raggiungibile. Da questo momento il kernel accetta connessioni TCP sulla porta esposta e Kubernetes marca il Pod come `Running`. Questo stato riflette solamente il punto di vista dell'infrastruttura, non c'è alcuna garanzia che il processo applicativo sia realmente pronto.
- L'**application readiness** è invece la condizione in cui il processo applicativo ha completato la propria inizializzazione e sta attivamente servendo richieste. A seconda del linguaggio e del framework, questa fase può includere: l'avvio di una macchina virtuale (nel caso della JVM) oppure l'inizializzazione di un contesto applicativo come Spring.

Il gap temporale Esiste sempre un intervallo in cui il Pod è *infrastructure-ready* ma non *application-ready*: il TCP handshake riesce (gestito dal kernel), ma nessuna risposta HTTP arriva, semantica tipica di eccezioni come `NoHttpResponseException`, connessione stabilita a livello di trasporto, ma nessuna risposta applicativa.

La **readiness probe** colma questo gap, finché l'applicazione non risponde all'health check, il Pod non riceve traffico. Kubernetes affianca la **liveness probe**, con semantica diversa ("il processo è vivo?"): se fallisce ripetutamente, riavvia il container per rilevare deadlock o stati irrecuperabili.

La distinzione è netta, un container può essere vivo ma non pronto. Oppure può smettere temporaneamente di esserlo senza dover essere riavviato.

La presenza di entrambi i meccanismi conferma che la separazione tra infrastruttura e applicazione non è risolvibile a livello di rete senza cooperazione dell'applicazione stessa.

Anche con la readiness probe configurata, lo stato `Running` del Pod non garantisce la disponibilità dell'endpoint HTTP: scenari come testing automatizzato o

script di deploy richiedono una verifica esplicita dello stato applicativo, ottenibile con `kubectl wait`, che blocca l'esecuzione finché una condizione specifica non è soddisfatta.

In alternativa, si può implementare un polling esplicito su un endpoint di health check dal lato client, trasformando un'assunzione implicita in un'asserzione verificabile.

Horizontal Pod Autoscaler Per adattare dinamicamente il numero di repliche al carico, Kubernetes offre l'**Horizontal Pod Autoscaler** (HPA). L'HPA monitora metriche come il consumo di CPU o memoria e scala automaticamente i Pod tra un minimo e un massimo predefiniti, creando o terminando repliche in base a soglie configurabili. [32]

Esecuzione Locale Nanofaas permette di sostituire l'utilizzo di Kubernetes con Docker, delegando la gestione dei container locali. Kubernetes presenta molti componenti, quasi del tutto inutili se si vuole testare semplicemente il sistema. Avviare il control plane, aggiungere l'astrazione dei Pod con il loro networking è eccessivo e troppo costoso dal punto di vista computazionale e temporale.

2.5 Il ruolo di Git

La metodologia DevOps richiede un sistema di controllo versione per annullare le modifiche e ripristinare stati precedenti del software.

Git, tra i VCS (Version Control Software), si distingue per il modello di gestione dei dati: anziché registrare differenze progressive, salva a ogni commit uno *snapshot* dello stato del progetto, invece che duplicare gli elementi rimasti invariati. [29]

Nella prima immagine si può vedere come funzionava un VCS prima di Git.

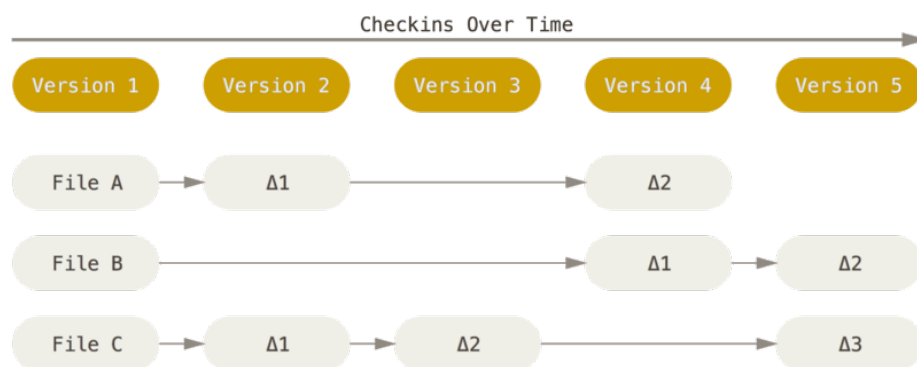


Figura 2.1: VCS-Generico

Git segue invece questa struttura:

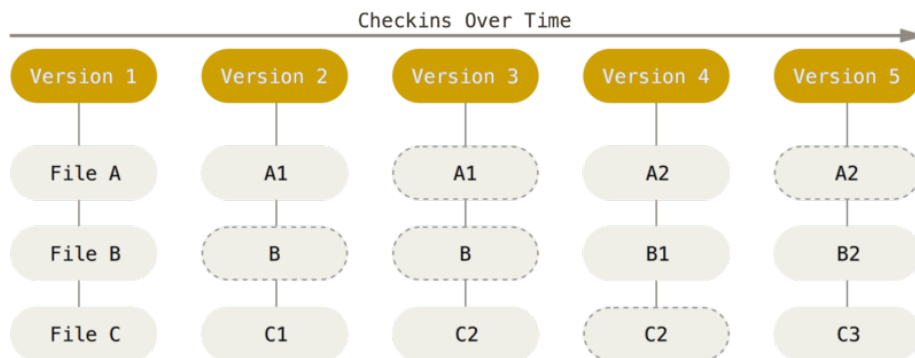


Figura 2.2: Git

GitHub GitHub è uno dei tanti strumenti usati sia negli ambienti cloud native e sia nel progetto Nanofaas. È il più grande distributore di repository Git. [19]

Issue Le *issue* sono segnalazioni che chiunque può aprire su una repository pubblica. Non sono pensate esclusivamente per gli sviluppatori del progetto: chiunque, anche un utente esterno, può usarle per segnalare un malfunzionamento, proporre un miglioramento o chiedere chiarimenti sul funzionamento del codice.

Capitolo 3

Nanofaas

Nanofaas è una piattaforma *Function-as-a-Service* (FAAS) progettata per l'esecuzione di funzioni *serverless* su infrastruttura Kubernetes. Il progetto nasce con l'obiettivo di offrire un sistema leggero, modulare e orientato alle prestazioni, in cui tutte le componenti di controllo risiedono in un unico processo, il *control plane*, mentre l'esecuzione delle funzioni avviene in Pod separati, creati dinamicamente su richiesta.

Il modello FAAS adottato segue il paradigma consolidato: lo sviluppatore registra una funzione specificando l'immagine del container, le risorse necessarie e i parametri di configurazione; il sistema si occupa poi di ricevere le richieste, instradarle al container che esegue la funzione e restituire il risultato. Ciò che distingue nanofaas da altre piattaforme FAAS è la scelta di mantenere l'intero *control plane* in un singolo Pod, senza dipendere da code esterne, database o sistemi di coordinamento distribuito.

Obiettivi di progettazione Il progetto si fonda su quattro pilastri fondamentali:

1. **Performance come priorità assoluta.** Ogni decisione architetturale privilegia la riduzione della latenza: strutture dati in memoria, gestione non bloccante delle richieste e ottimizzazioni mirate a ridurre il *cold start overhead* e il tempo di risposta.
2. **Architettura a singolo Pod.** Il *control plane* non prevede replicazione orizzontale, quindi API gateway, registry, code e scheduler, coesistono in un unico processo. Questo semplifica la gestione dello stato (non serve coordinamento distribuito) ed elimina le latenze di rete interne, a costo di sacrificare l'alta disponibilità.
3. **Modularità a tempo di compilazione.** Le funzionalità opzionali non sono attivate da flag a runtime, ma caricate come moduli separati al momento della compilazione. Questo garantisce che nessun codice inutilizzato venga incluso nell'artefatto finale (aspetto rilevante per le compilazioni native) e che le dipendenze tra moduli siano esplicite e verificabili staticamente.

4. **Esecuzione in isolamento.** Ogni funzione viene eseguita in un Pod Kubernetes dedicato, garantendo isolamento a livello di processo, risorse e rete. Le funzioni non condividono memoria né file system con il *control plane*.

Non-obiettivi Nanofaas, nella sua versione attuale, non implementa:

- Alta disponibilità → il control plane è un punto singolo di fallimento e un riavvio causa perdita dello stato in memoria.
- Code durevoli → nessuna persistenza su disco o message broker esterni.
- Autenticazione e autorizzazione → la sicurezza è delegata al layer infrastrutturale Kubernetes.
- Multi-tenancy avanzata.

3.1 Panoramica dell'architettura

L'architettura di nanofaas si sviluppa su due livelli, un *control plane* e dei *Pod* effimeri.

I componenti del control plane Il *control plane* è organizzato in componenti con responsabilità ben definite. L'**API Gateway** è il punto di ingresso REST per la registrazione e l'invocazione delle funzioni e gestisce rate limiting, mappatura degli errori in codici HTTP e tracciamento distribuito. Il **Function Registry** mantiene in memoria le specifiche di tutte le funzioni registrate (nome, immagine, risorse, timeout, concorrenza, dimensione coda, tentativi). Due moduli opzionali gestiscono le code di invocazione: la coda asincrona (**async-queue**) a capacità limitata per funzione e il controllo di ammissione sincrono (**sync-queue**) che stima i tempi di attesa e respinge preventivamente le richieste. Lo **Scheduler** è un thread dedicato che preleva le richieste dalle code e le inoltra al **Dispatcher Router**, che seleziona il dispatcher appropriato in base alla modalità di esecuzione configurata (LOCAL, POOL o DEPLOYMENT). L'**Execution Store** infine archivia in memoria i record di esecuzione con stato, timestamp e risultato.

Il Pod di esecuzione Ciascuna funzione viene eseguita in un Pod Kubernetes separato che contiene due componenti. Il **Watchdog** è un eseguibile Rust (2 MB) che funge da supervisore: riceve la richiesta HTTP, la inoltra al processo utente tramite proxy HTTP o stdin/stdout e restituisce il risultato al control plane. In modalità *warm*, il watchdog mantiene attivo il processo tra un'invocazione e l'altra. Il **Runtime utente** è il codice della funzione vera e propria, eseguibile in Java, Python, Go, JavaScript o come binario generico. L'interfaccia tra watchdog e runtime è standardizzata in JSON e indipendente dal linguaggio. [6], [12]

3.2 Il Control Plane

Perché un control plane personalizzato? Il control plane di Kubernetes (API Server, Controller Manager, Scheduler) non è ottimizzato per invocazioni serverless dal ciclo di vita brevissimo. Interrogare l'API Server di Kubernetes per ogni singola richiesta introdurrebbe latenze inaccettabili e un carico eccessivo sul cluster. Nanofaas introduce quindi un proprio control plane specializzato in memoria, il cui scopo è gestire il routing HTTP veloce, l'accodamento e la gestione sincrona delle risposte. [11]

Il confine di responsabilità Nanofaas delega a Kubernetes tutto ciò che non è strettamente legato al percorso critico dell'invocazione.

- **Scheduling e allocazione risorse:** è il kube-scheduler a decidere su quale nodo eseguire i Pod, basandosi su CPU e memoria richieste.
- **Networking e isolamento:** il routing TCP/IP e il DNS interno sfruttano il networking nativo di Kubernetes, senza layer di rete custom.
- **Gestione dei container:** lo scaricamento delle immagini e l'avvio effettivo (cgroups, namespaces) sono delegati al kubelet.

nanofaas NON delega a Kubernetes:

- **Routing L7 e API Gateway:** il punto di ingresso HTTP, la mappatura degli URL alle funzioni e il rate limiting sono gestiti internamente.
- **Code in memoria:** l'accodamento per gestire i picchi di carico e la back-pressure vive esclusivamente nel control plane (Kubernetes non ha code native di richieste HTTP). [7]
- **Pool management:** per le esecuzioni *warm*, nanofaas mantiene internamente la mappa dei Pod liberi/occupati, instradando le richieste direttamente al Pod libero senza passare dal bilanciatore standard di Kubernetes.
- **Autoscaling reattivo:** Nanofaas scala basandosi sulla lunghezza delle proprie code in memoria, reagendo più velocemente dell'Horizontal Pod Autoscaler standard di Kubernetes.

3.2.1 API Gateway

L'API Gateway è il punto di ingresso unico per tutte le interazioni con la piattaforma ed espone un'API RESTful organizzata in tre aree funzionali.

Per la **gestione delle funzioni** sono disponibili endpoint per elencare, registrare, recuperare, rimuovere e scalare le funzioni. La registrazione avviene tramite un descrittore YAML (`function.yaml`) che specifica nome, immagine, risorse, timeout, concorrenza, dimensione coda e tentativi, il sistema applica valori predefiniti ai campi non specificati.

Per l'**invocazione** sono disponibili due modalità: sincrona, in cui il client attende il risultato (potenziali risposte: 200 OK, 408 Timeout, 429 Troppe richieste, 500 Errore interno) e asincrona, in cui il client riceve immediatamente un identificativo di esecuzione (202 Accepted) per interrogare lo stato successivamente.

È inoltre presente un'API amministrativa opzionale di **configurazione a caldo** che consente di modificare parametri operativi (rate limit, profondità code, soglie di ammissione) senza riavviare il control plane.

3.3 Architettura modulare

Uno degli aspetti più caratteristici di nanofaas è la sua architettura modulare, basata su componenti opzionali inclusi durante la fase di compilazione. A differenza dei tradizionali *feature flag* a runtime, in cui il codice delle funzionalità disabilitate rimane comunque presente nell'applicazione, nanofaas utilizza un approccio di *compile-time composition* dove i moduli non selezionati vengono completamente esclusi dall'artefatto finale. Questo permette di ottenere un eseguibile più leggero, ridurre le dipendenze e limitare la superficie di attacco dell'applicazione.

3.3.1 Meccanismo di caricamento

Il sistema si basa su un'interfaccia comune implementata da ciascun modulo opzionale. Il caricamento avviene tramite il meccanismo standard `ServiceLoader` di Java: ogni modulo dichiara la propria implementazione in un file di metadata all'interno del proprio JAR e all'avvio il `ServiceLoader` scansiona il classpath istanziando automaticamente i moduli disponibili. La selezione avviene tramite un profilo Gradle o una variabile d'ambiente.

I moduli non elencati non vengono nemmeno compilati. [8]

Per garantire il funzionamento anche senza moduli, il nucleo definisce implementazioni fittizie (*no-op*) per le interfacce sostituibili dai moduli (accodamento, metriche di scaling, ammissione sincrona, validazione immagini), che vengono automaticamente rimpiazzate dalle implementazioni reali quando i moduli sono caricati.

3.3.2 Moduli principali

async-queue → Introduce code a capacità limitata per funzione. Un gestore centralizzato mantiene lo stato di ciascuna coda, con un meccanismo a slot per limitare il numero di invocazioni simultanee. Uno scheduler asincrono dedicato preleva le richieste e le inoltra al dispatcher, con un criterio di *equità* (massimo due richieste per funzione per ciclo) per evitare che funzioni con code profonde monopolizzino le risorse.

sync-queue → Aggiunge un controllo di ammissione per le invocazioni sincrone, respingendo le richieste quando la coda è piena o quando il tempo di attesa stimato supera una soglia. In caso di rifiuto, il client riceve il codice 429

con header che indicano quando ritentare e il motivo del rifiuto. Un thread dedicato gestisce le richieste ammesse con backoff adattivo e salto delle funzioni bloccate, evitando *head-of-line blocking*.

Moduli di Deployment → Due backend alternativi: uno basato su Kubernetes (crea Deployment, Service e HPA nativi) e uno basato su container runtime locali (Docker/Podman per ambienti senza Kubernetes).

Altri moduli → Esistono moduli per la validazione delle immagini container, per la configurazione a caldo dei parametri operativi e per l'esposizione di metadati di build (versione, hash Git, moduli caricati).

3.4 Modalità di esecuzione

Nanofaas definisce tre modalità di esecuzione che determinano come il control plane interagisce con il container della funzione. La scelta è specificata al momento della registrazione e può essere soggetta a degradazione automatica se la modalità richiesta non è disponibile.

Il codice supporta rigorosamente tre valori tecnici: LOCAL, POOL e DEPLOYMENT. All'atto pratico, DEPLOYMENT e POOL rappresentano semplicemente le implementazioni fisiche del concetto teorico "WARM".

3.4.1 LOCAL

La modalità più semplice: il dispatcher non contatta alcun container esterno, ma restituisce direttamente l'input ricevuto. È pensata esclusivamente per il testing dell'infrastruttura del control plane senza richiedere codice utente né risorse Kubernetes.

3.4.2 POOL

Presuppone l'esistenza di container già in esecuzione (pool *warm*) accessibili tramite un URL noto. Il control plane non gestisce il ciclo di vita di questi container: provisioning, scaling e monitoraggio sono responsabilità esterne. Al momento dell'invocazione, il dispatcher effettua una richiesta HTTP diretta al container e interpreta la risposta come risultato.

3.4.3 DEPLOYMENT

Il caso d'uso più completo: il control plane gestisce l'intero ciclo di vita dei container. Al momento della registrazione crea le risorse Kubernetes necessarie (Deployment, Service, eventuale HPA), attende che i Pod siano pronti e memorizza l'URL dell'endpoint. Da quel momento il dispatch procede come in modalità POOL, ma il control plane mantiene il controllo sul numero di repliche e può scalarle dinamicamente. La rimozione della funzione comporta la distruzione di tutte le risorse. La modalità può operare sia su Kubernetes che su container runtime locali.

3.5 Flusso di esecuzione delle invocazioni

Il percorso di una richiesta dal client al risultato è il cuore operativo della piattaforma. Nanofaas supporta invocazioni sincrone e asincrone, che condividono la stessa infrastruttura ma differiscono nel contratto verso il client.

3.5.1 Invocazione sincrona

L'invocazione sincrona (POST /v1/functions/{name}:invoke) mantiene il client in attesa fino al completamento. Il flusso si articola in sei fasi:

1. **Rate limiting:** verifica che la richiesta non superi la soglia globale (in caso contrario, HTTP 429 immediato).
2. **Risoluzione:** recupera la funzione dal registry (se assente, HTTP 404).
3. **Creazione esecuzione:** genera un identificativo e un record di esecuzione. Se il client ha fornito una chiave di idempotenza, il sistema restituisce immediatamente il risultato di un'esecuzione precedente già completata.
4. **Ammissione:** a seconda dei moduli caricati, la richiesta viene valutata dal controllo di ammissione sincrono (respingendo con 429 se la coda è piena o il tempo stimato è eccessivo), accodata nella coda asincrona per-funzione (429 se piena), oppure inviata direttamente al dispatcher in assenza di moduli.
5. **Dispatch:** il dispatcher inoltra la richiesta al container della funzione tramite HTTP POST e attende la risposta.
6. **Completamento e retry:** in caso di successo il client riceve HTTP 200; in caso di errore con tentativi residui la richiesta viene riaccodata; in caso di errore terminale o timeout il client riceve HTTP 500 o 408.

Il client attende in modo non bloccante durante l'intero flusso.

3.5.2 Invocazione asincrona

L'invocazione asincrona (POST /v1/functions/{name}:enqueue) richiede il modulo `async-queue`. Dopo rate limiting e risoluzione della funzione, la richiesta viene accodata e il client riceve immediatamente HTTP 202 con l'identificativo dell'esecuzione. Il client può interrogare lo stato tramite GET /v1/executions/{id}, che restituisce lo stato corrente e, se disponibile, il risultato.

3.5.3 Gestione dei tentativi e timeout

Il numero massimo di tentativi in caso di errore è configurabile per funzione. Dopo ogni fallimento non terminale, la richiesta viene riaccodata. Il timeout dell'invocazione (con un default di 30 secondi) è applicato lato control plane: se la funzione non completa entro il tempo configurato, l'esecuzione viene marcata come timeout in modo terminale.

3.6 I runtime delle funzioni

Il Watchdog in Rust agisce come processo principale (PID 1) all'interno del container della funzione ed è l'unico punto di contatto verso il control plane. I vari runtime non comunicano mai direttamente con l'esterno ma girano supervisionati dal Watchdog, che instrada le richieste in due modalità:

- proxy HTTP per linguaggi con buona concorrenza asincrona (Java, Go)
- iniezione via stdin/stdout per linguaggi di scripting (Python, Node.js)

Questo tipo di progettazione separa la logica di networking e gli health-check (centralizzati nel Watchdog) dal codice utente puro (eseguito nei singoli runtime). Tutti i runtime condividono lo stesso contratto dati JSON.

3.6.1 Java Runtime

Il runtime Java è un server HTTP minimale che espone l'endpoint di invocazione e gli health check. Lo sviluppatore implementa un'interfaccia standard: il runtime la scopre automaticamente all'avvio e se sono presenti più implementazioni una variabile d'ambiente specifica quale utilizzare. Il runtime supporta la compilazione nativa tramite GraalVM, che produce un eseguibile ad avvio quasi istantaneo e consumo di memoria ridotto, aspetto fondamentale per lo scenario FAAS dove il cold start è uno dei principali fattori di latenza. [6], [10]

3.6.2 Python Runtime

Il runtime Python è un'applicazione Flask che supporta l'esecuzione *warm* con container persistente tra invocazioni. Il codice utente viene caricato dinamicamente tramite variabili d'ambiente che specificano modulo e funzione da invocare. Il runtime supporta sia esecuzione *warm* che one-shot e implementa un meccanismo di callback asincrono con retry automatico verso il control plane dopo il completamento dell'esecuzione.

3.6.3 SDK per sviluppatori

Nanofaas mette a disposizione SDK in quattro linguaggi: **Java** (basato su Spring), **Java Lite** (privo di dipendenze Spring, ottimizzato per compilazioni native), **Go** e **JavaScript** (per Node.js). Tutti implementano lo stesso contratto e garantiscono un'interfaccia di sviluppo uniforme.

3.7 Osservabilità

L'osservabilità della piattaforma si basa su tre pilastri. Il *control plane* espone **health check** standard (liveness e readiness) sulla porta 8081, utilizzati dai probe di Kubernetes; anche i runtime delle funzioni espongono i propri endpoint di health. Le **metriche Prometheus** sono esposte dal control plane e coprono lo stato di code, esecuzioni e concorrenza. Il **tracciamento distribuito** è implementato tramite un header HTTP che il control plane genera e propaga a tutte

le componenti coinvolte nell'invocazione; i log strutturati includono identificativo esecuzione, nome funzione, trace ID, tentativo e stato, consentendo di correlare gli eventi di una singola invocazione attraverso l'intero sistema. [9]

Capitolo 4

Metodologia di Esplorazione e Valutazione

4.1 Approccio iniziale alla piattaforma

Inizialmente l'obiettivo della tesi era "generico" e consisteva nell'esplorare informalmente la piattaforma, verificando che almeno si potesse scaricare da GitHub, che si avviasse e che si potessero usare i comandi base. Queste primissime interazioni non sono state salvate, solo dopo, alcune sessioni del terminale sono state salvate tramite file .txt.

4.2 Documentazione rapida dei comandi

Durante l'esplorazione "informale" della piattaforma è stato subito necessario cominciare a **riassumere** il lavoro svolto per:

- Ottenere velocemente i comandi esatti da usare in determinate situazioni.
- Non doversi ricordare a memoria tutti i comandi, che fin da subito hanno cominciato a moltiplicarsi.
- Rimediare alla scarsa documentazione del progetto, che a volte era assente e a volte non era intuitiva.

Sono state quindi scritte delle documentazioni personali, meno verbose e più pratiche di quelle fornite da nanofaas. [33]

Realizzazione Queste documentazioni personali sono state **fondamentali** per il passaggio successivo. Infatti durante la generazione di questi "riassunti" si è finalmente compreso la necessità di strutturare meglio l'esplorazione della piattaforma, usando come appoggio libri di testo e soluzioni usate in contesti lavorativi.

4.3 Fine dell'esplorazione informale

Problemi dell'esplorazione informale Nella fase iniziale ciò che veniva testato e come effettivamente veniva testato è stato poco strutturato. Tra i problemi più importanti che sono avvenuti in questa fase:

- Presenza di configurazioni precedenti che giravano in background
- Test di alcune funzionalità sempre sulla stessa configurazione, spesso modificata e quindi differente dalla versione ufficiale.
- I problemi trovati non venivano immediatamente segnalati.

Exploratory Testing È stata fatta una ricerca online per capire quale approccio scegliere, che avesse un'adeguata coerenza con gli obiettivi della tesi. È stato scelto il libro: "Explore It!: Reduce Risk and Increase Confidence With Exploratory Testing", scritto da Elisabeth Hendrickson. [2]

Definizione Il test esplorativo non ha come obiettivo l'esecuzione di script che testino il codice ogni volta che viene effettuata una modifica.

Charter Questa è stata la parte più importante. I Charter sono delle missioni/obiettivi con il fine di esplorare una funzionalità specifica della piattaforma

Issue GitHub I problemi trovati durante le esplorazioni sono stati segnalati su GitHub attraverso le issue, in modo da tracciare e catalogare tutti i problemi trovati.

4.4 Esplorazione Automatizzata

Nonostante l'approccio di 'Explore it!' di Elisabeth Hendrickson sia più strutturato, non risolve il problema delle enormi sessioni del terminale, poco leggibili e difficili da riprodurre.

Per questo la digitazione sul terminale è stata sostituita con degli script bash. Il motivo per cui è stato usato bash è per "simulare" la digitazione dal terminale. Gli script non sono stati pensati propriamente come test e2e o test unitari. Per questo non sono stati usati linguaggi come Python.

Inoltre, dopo le varie esperienze con l'esplorazione informale, alcuni pattern di comandi hanno cominciato ad affiorare, giustificando ancora di più la creazione di script.

Architettura degli script Le documentazioni prodotte durante le precedenti sessioni di lavoro sono state usate come base degli script. È stato usato "justfile" per organizzare meglio gli script. [21]

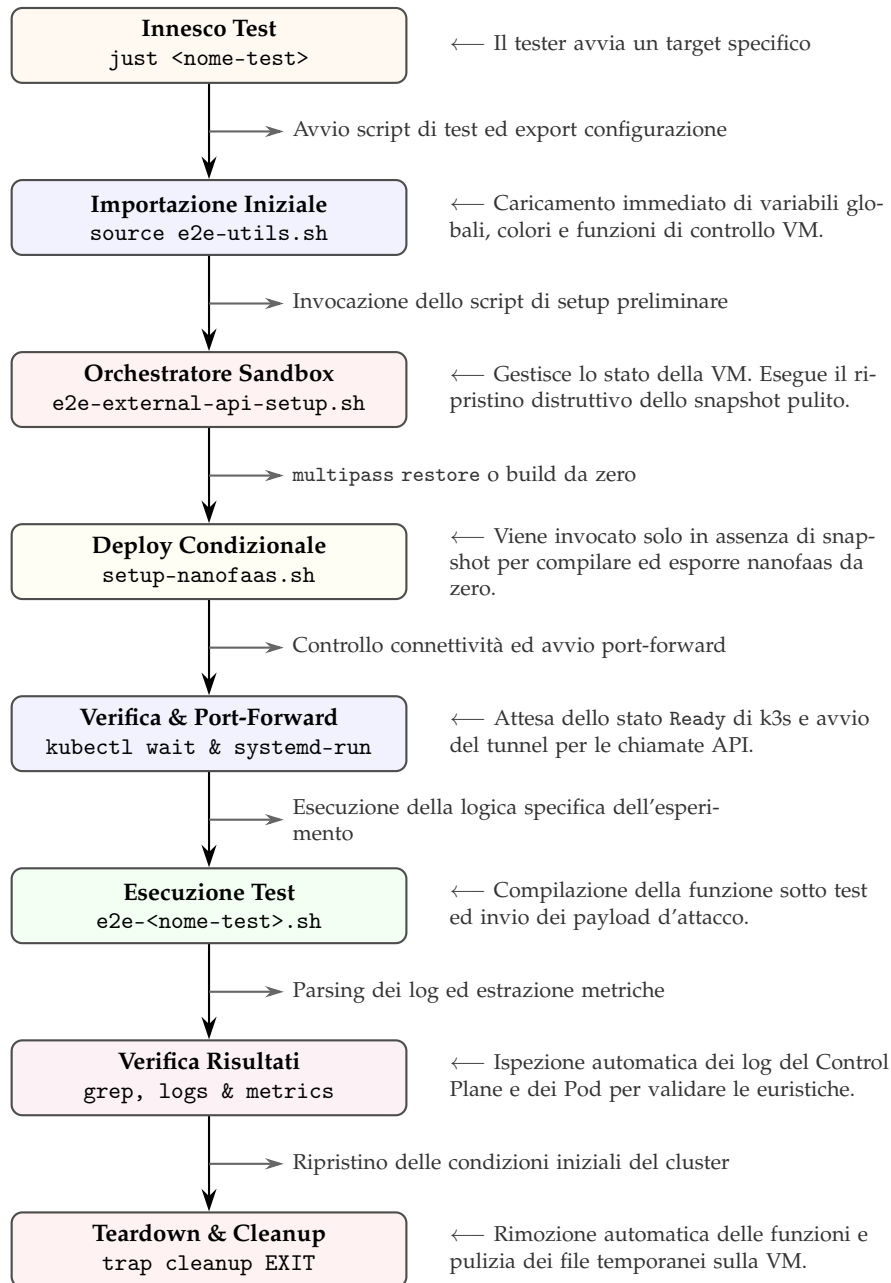


Figura 4.1: Ciclo di vita completo di una sessione di test esplorativo assistito da strumenti

L'automazione con gli script non è stata quindi utilizzata per sostituire l'attività esplorativa, ma come strumento per garantire condizioni iniziali controllate, riducendo l'influenza di fattori esterni e aumentando l'affidabilità delle osservazioni raccolte. Questo è dovuto in parte anche alla natura di Kubernetes e nanofaas. Sono infatti ambienti molto complessi, con tante componenti che interagiscono tra di loro. È difficile testare "facilmente" ambienti del genere. Questo approccio è anche chiamato: Tool-Supported Exploratory Testing. [2]

4.5 Vantaggi del nuovo approccio

- Nessuna configurazione "vecchia" che gira in background
- Garanzia di una configurazione "sempre uguale" ed effimera, ossia che non influenzerà le configurazioni future.
- Permette di ricordarsi meglio cosa è stato scritto e cosa è stato testato.
- Alcune componenti del test richiedono necessariamente loop, controlli e molte righe di codice da scrivere.

Capitolo 5

Risultati dell'analisi

5.1 Premesse sui risultati

In questo capitolo si discutono i risultati rilevanti. Esplorazioni ed analisi che si sono rivelate un successo fin da subito non vengono discusse.

5.1.1 Mancanza di commenti al codice

La codebase di nanofaas presenta pochissimi commenti all'interno del codice. Questa mancanza, aggravata da una documentazione carente, ha reso estremamente difficile lavorare con la piattaforma.

È stato quindi fondamentale l'utilizzo di sistemi IA per analizzare il codice e capire che ruolo avessero i componenti nell'architettura. [26]

5.2 Esecuzione dei test

Per iniziare la valutazione della piattaforma, una delle prime attività ha riguardato l'esecuzione di tutti i test forniti dalla piattaforma nanofaas.

5.2.1 Avvio della Piattaforma

La valutazione della piattaforma è stata effettuata al di fuori dell'ambiente dove è stata sviluppata (macOS). Questo ha comportato subito un problema.

La configurazione predefinita di Fedora prevede l'uso di `firewalld` come firewall dinamico. `firewalld` è un servizio che decide quali connessioni di rete sono permesse e quali vanno bloccate. Per prendere queste decisioni, il firewall raggruppa le interfacce di rete (Wi-Fi, ethernet, bridge, ecc.) in diverse **zone**, ciascuna con un insieme di regole.

Le regole bloccano il traffico di rete proveniente dall'esterno e diretto verso la macchina stessa. Questo significa che quando un programma su un altro dispositivo (o su una macchina virtuale) cerca di connettersi a un servizio in esecuzione nel computer, `firewalld` respinge la connessione.

In particolare, ogni volta che Multipass (collegata tramite l'interfaccia bridge

mpqemubr0) provava a comunicare verso l'esterno o verso l'host, veniva bloccata.

Questo ha causato il fallimento di tutti i test E2E, impossibilitati a connettersi a Internet per le prime operazioni di bootstrap della piattaforma (apt per i pacchetti, pull delle immagini dei container, download di k3s, ecc.).

La soluzione adottata è stata quella di associare l'interfaccia mpqemubr0 alla zona `trusted` di `firewalld`, che accetta tutto il traffico senza filtraggio:

```
$ sudo firewall-cmd --permanent --zone=trusted --add interface=mpqemubr0
$ sudo firewall-cmd --reload
```

Con la nuova configurazione, le regole di forward verso mpqemubr0 accettano il traffico in entrambe le direzioni.

5.2.2 Test e2e-k8s-vm

Uno script in particolare ha causato problemi: 'e2e-k8s-vm.sh'. lo script funziona nella maniera seguente:

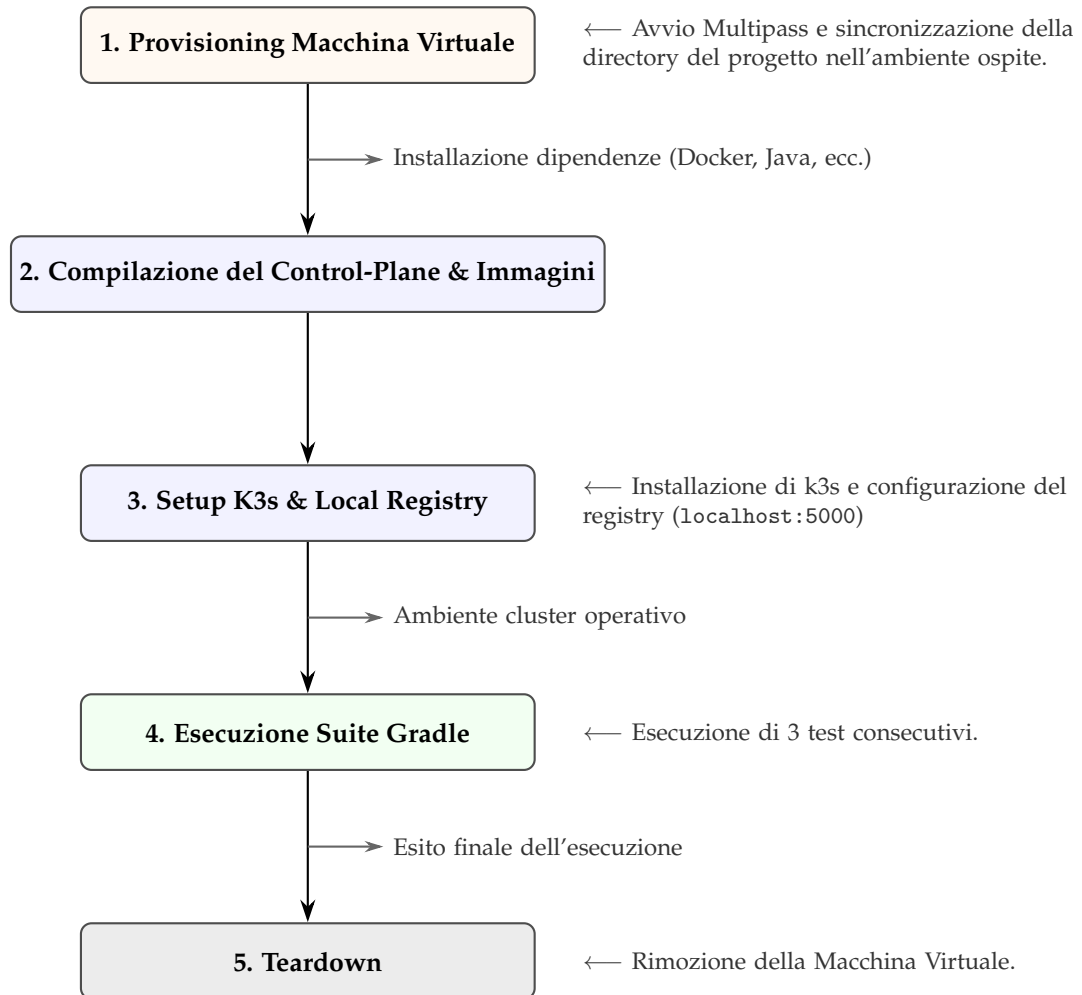


Figura 5.1: script `e2e-k8s-vm.sh`

Durante l'esecuzione della suite E2E, lo script avvia tre test in sequenza:

- **k8sRegisterInvokeAndPoll**: registra una funzione, la invoca, viene creato un container, la funzione viene eseguita e poi il test valuta se il risultato è corretto.
- **k8sColdStartMetrics_areRecorded**: Simula una richiesta a una funzione senza Pod attivi. Assicura che il Control Plane forzi l'avvio del container (cold start), gestisca l'attesa e che le metriche di latenza vengano propriamente registrate

- **k8sSyncQueueBackpressure:** Invia un burst di chiamate sincrone per saturare il Control Plane. Verifica che il sistema prevenga il collasso accodando correttamente le richieste o applicando backpressure.

Fallimento Durante l'esecuzione sono falliti due test:

- `k8sColdStartMetrics_areRecorded()`
- `k8sSyncQueueBackpressure()`

Il fallimento dei due test è stato, in realtà, causato dal primo. Infatti **k8sRegisterInvokeAndPoll** ha causato problemi al Control Plane, risultando in un crash di quest'ultimo. Le cause di questo crash non sono state trovate.

Ad ogni modo il Control Plane viene riavviato, però **e2e-k8s-vm.sh** non si assicura che i test successivi abbiano le risorse necessarie per essere eseguiti. Dunque i due test sono stati avviati con un Control Plane che si stava ancora riavviando, causando ovviamente un errore. Infatti entrambi i test hanno restituito un errore di **NoHttpResponseException** che implica una porta aperta senza però scambiarsi messaggi, che è ben diverso da un errore 'ConnectionRefusedException', dove nessuno risponde alle richieste o la porta è chiusa.

Soluzione Per risolvere, prima di ogni test si esegue un loop di controllo dell'endpoint di Spring Actuator finché non risponde `"status":"UP"`.

- `awaitHealth(porta-control-plane, "/actuator/health")`
- `awaitHealth(porta-function-runtime, "/actuator/health")`

Solo a questo punto il test viene eseguito. In questo modo il test non è in grado di avviarsi finché il Pod non è pronto (nel caso sia avvenuto un crash).

Il risultato di questa valutazione è stato un successo, perché è stato individuato un problema in un test e2e fornito da nanofaas che probabilmente non si verificava all'interno dell'ambiente di produzione.

Successivamente, a seguito di un refactoring della codebase di nanofaas, lo script `e2e-k8s-vm.sh` è stato rimosso; tuttavia, l'analisi svolta conserva la sua validità metodologica nell'evidenziare criticità gestionali. [23]

5.2.3 Cluster Kubernetes mancante

La configurazione del cluster ha presentato ulteriori problematiche. In particolare, l'esecuzione del comando `./gradlew test` richiedeva la presenza di un cluster k3s attivo.

Nelle fasi iniziali dello studio, i test venivano eseguiti in assenza di servizi di orchestrazione attivi, a causa di una documentazione non esaustiva in merito ai requisiti di esecuzione.

```
# Esegue tutti i test, eccetto alcuni test e2e
./gradlew test
```

All'interno di Gradle esiste un controllo, che se il tag "runE2e" non è impostato, allora bisogna escludere tutti i test contrassegnati 'inter e2e'. Questo esclude tutti i test complessi che richiedono più componenti o un cluster Kubernetes attivo.

Ad ogni modo, il test non fallisce brutalmente se non si ha un cluster attivo. Infatti gli script all'interno di nanofaas sono in grado di creare macchine virtuali multipass e configurare un cluster con dentro nanofaas.

I prerequisiti per avviare K8sE2eTest sono:

- KUBECONFIG → È una variabile d'ambiente utilizzata per trovare il file di configurazione con le credenziali e gli indirizzi necessari per connettersi a un cluster Kubernetes attivo.
- Un namespace all'interno della macchina virtuale Multipass
- Un Control Plane e un Function Runtime già avviati

All'interno di K8sE2eTest, nella fase di inizializzazione si implementa un controllo, se la variabile d'ambiente KUBECONFIG non è configurata o non punta a un file di configurazione Kubernetes valido, il test viene ignorato, permettendo di completare gli altri test senza avere Kubernetes installato.

Però, questo non è stato sufficiente a far saltare il test. La variabile KUBECONFIG infatti può essere impostata come globale e quindi il test potrebbe riconoscerla come "già configurata". In tale scenario, le precondizioni del test vengono soddisfatte con esito positivo.

Questo errore permette al test di continuare l'esecuzione, che ovviamente fallisce a causa di un cluster Kubernetes inesistente.

```
# Ricerca del percorso della variabile
printenv KUBECONFIG
# Il comando ha restituito: /home/luca/.kube/config

# Visualizzazione del file
bat /home/luca/.kube/config
```

Ecco cosa è stato visualizzato:

```
apiVersion: v1
contexts:
- context:
    cluster: kind-nanofaas-local
    user: kind-nanofaas-local
    name: kind-nanofaas-local
```

È molto probabile che questa configurazione sia stata generata con "kind", in un tentativo di avviare la piattaforma durante una sessione di esplorazione

precedente.

Questo errore ha spinto a terminare definitivamente l'esplorazione della piattaforma nanofaas in un ambiente **non** isolato.

5.3 Modalità Sviluppo in Locale

La modalità di sviluppo in locale presuppone che Kubernetes non venga utilizzato, per far spazio invece ad una modalità più leggera e veloce da usare, ideale per testare se le funzioni vengono almeno compilate ed eseguite. [24] [25]

Control Plane e Function Runtime In Nanofaas questi due componenti sono separati. Più precisamente, il Function Runtime serve unicamente a passare l'input alla funzione e restituire il risultato. Il control plane si occupa solo di ricevere le richieste dell'utente e gestirle.

Questa separazione permette di vedere i due componenti come due processi diversi, con le loro porte.

Comunicazione tra i due componenti La comunicazione tra Control Plane e Function Runtime avviene tramite chiamate HTTP/REST. L'intera comunicazione è un semplice scambio di messaggi JSON. Questo permette di "staccare" la comunicazione dalle logiche di Kubernetes o Docker.

Modalità di esecuzione delle funzioni Dentro il Control Plane c'è l'API Gateway, il Queue Manager, lo Scheduler e il dispatcher. Quest'ultimo "decide" il luogo dell'esecuzione della funzione:

- **WARM** → Il dispatcher cerca dei container già avviati su Kubernetes e inoltra la richiesta al primo libero.
- **LOCAL** → Il dispatcher esclude completamente Kubernetes. Diventa un semplice router HTTP. Legge l'url (localhost) e inoltra la richiesta direttamente a quell'indirizzo.
- **JOB** → Il dispatcher contatta l'API Server di Kubernetes ordinando di creare un nuovo Pod (Job) e di scaricare l'immagine Docker della funzione. Questa modalità però è solo descritta nella documentazione. Non è stata ancora implementata.

La modalità LOCAL quindi ha permesso di testare inizialmente le varie funzioni della piattaforma nanofaas senza avviare un cluster Kubernetes. Dunque per eseguire nanofaas in locale, è necessario:

Avviare il Control Plane Viene avviato sulla porta 8080.

```
./gradlew :control-plane:bootRun
```

Inoltre sulla porta 8081 si avvia il servizio di metriche Prometheus.

Avviare la funzione La funzione viene avviata sulla porta 8080. Questo causa un conflitto con le porte. È necessario quindi specificare un'altra porta nel comando di avvio:

```
# Avvio di una funzione d'esempio wordStat
SERVER_PORT=8084 FUNCTION_HANDLER=wordStatsHandler ./gradlew
:examples:java:word-stats:bootRun
```

Il comando quindi avvia un unico processo, con all'interno sia il Function Runtime che la funzione.

5.3.1 Le funzioni in nanofaas

Una considerazione importante dell'ultimo comando mostrato è il FUNCTION HANDLER. Mentre il SERVER PORT ha un'utilità nell'essere specificato, per il FUNCTION HANDLER serve una spiegazione diversa.

In nanofaas le funzioni devono implementare al loro interno una classe chiamata FunctionHandler, per esempio:

```
import it.unimib.dataai.nanofaas.common.runtime.FunctionHandler;
import org.springframework.stereotype.Component;

@Component
public class MyCustomHandler implements FunctionHandler {
    @Override
    public Object handle(InvocationRequest request) {
        // Funzione...
    }
}
```

Nel caso di Java, Spring Boot scandisce il codice alla ricerca delle funzioni che implementano l'interfaccia FunctionHandler. Il problema è che ad ogni funzione corrisponde un FunctionHandler e l'SDK non sa quale scegliere.

Specificando quindi il corretto FunctionHandler da utilizzare, si evitano ambiguità.

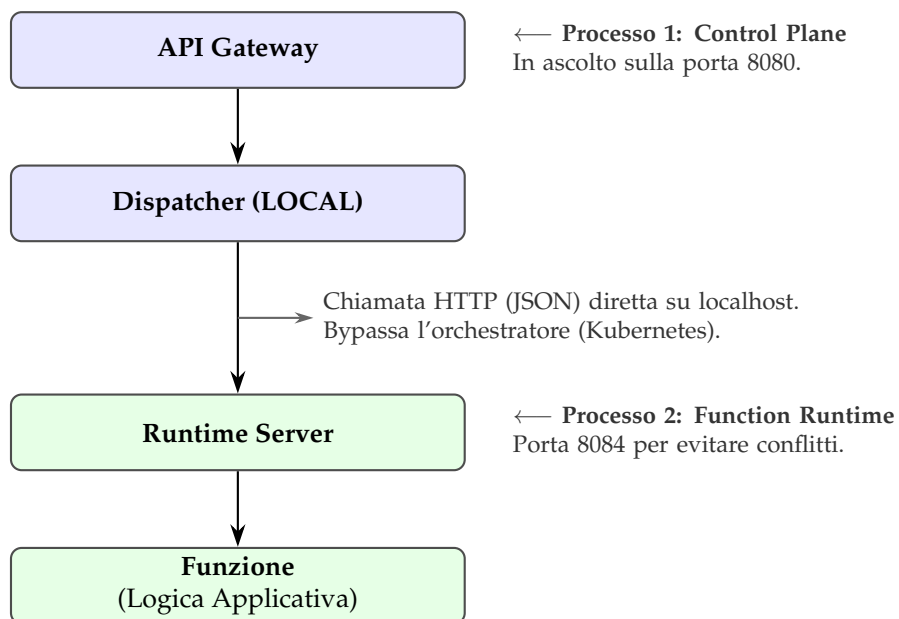


Figura 5.2: Architettura nanofaas: separazione dei processi in modalità LOCAL

Ad ogni modo queste conoscenze torneranno utili durante lo studio degli SDK della piattaforma.

5.4 Problemi di documentazione

Durante l'esplorazione della piattaforma più volte si sono verificati problemi riguardo la documentazione di nanofaas.

La documentazione, presente nel README del progetto o nella cartella /docs, è stata generata tramite intelligenza artificiale basandosi esclusivamente sull'esperienza d'uso in ambiente macOS e per utenti con conoscenza pregressa della piattaforma. Essa presenta omissioni relative a passaggi procedurali essenziali per utenti esterni. L'esplorazione di nanofaas all'interno di un ambiente nativo Linux ha sicuramente aiutato il progetto a rilevare molti errori.

5.4.1 Utilizzo di nanofaas-cli

Uno degli strumenti forniti da nanofaas è la cli. Questo strumento permette di eseguire comandi da terminale ed interagire con le funzioni registrate nella piattaforma.

Tra le prime sperimentazioni con questo strumento c'è stato fin da subito il desiderio di capire come inserire, compilare ed invocare una funzione dentro nanofaas. Una serie di azioni fondamentali, che rappresentano al meglio l'utilizzo "pratico" della piattaforma. Queste sono alcuni dei comandi che fin da subito erano disponibili nella cli.

```
# Mostra tutte le funzioni registrate
nanofaas fn list

# Mostra le specifiche della funzione
nanofaas fn get <name>

# cancella una funzione
nanofaas fn delete <name>

# crea o rimpiazza una funzione
nanofaas fn apply -f function.yaml
```

Una funzione fondamentale, quella dello scaffolding, non era presente. Testare le funzioni in nanofaas richiedeva inserire, compilare ed invocare le funzioni manualmente, digitando molte righe nel terminale. Tutto questo, sommato ad una documentazione scarna, ha reso l'esperienza poco piacevole.

È stata pertanto proposta l'estensione delle funzionalità della CLI e il miglioramento della documentazione, affiancati dall'introduzione di commenti esplicativi nel codice per chiarire le interazioni tra i diversi moduli.

5.4.2 Scaffolding

Lo scaffolding è stato implementato durante l'esplorazione della piattaforma. Si tratta di un tool Python che genera tutte le dipendenze e i file necessari per creare una funzione in nanofaas.

Supporta i linguaggi: Java, Python, Go, JavaScript e Bash.

```
# Esempio: creazione di una funzione in Java
./scripts/fn-init.sh funzione-esempio --lang java
```

Le principali azioni che compie fn-init sono:

- Genera il Dockerfile con la versione dell'immagine corretta
- Genera il file di build specifico per il linguaggio (build.gradle per Java, pyproject.toml per Python, go.mod per Go...)
- Genera il function.yaml per permettere la registrazione della funzione nella piattaforma
- Forza la creazione di ogni funzione a seguire le stesse convenzioni/strutture, rendendo l'intera piattaforma più coerente

Nel caso di java la struttura del progetto è la seguente:

```
funzione-esempio/
|-- src/
|   |-- main/java/.../
|   |   |-- Handler.java
|   |   |-- Application.java
|   |-- test/java/.../
|       |-- HandlerTest.java
|-- build.gradle
|-- Dockerfile
|-- function.yaml
|-- payloads/
|   |-- happy-path.json
|   |-- missing-input.json
```

Payloads Si può notare che viene creata anche una cartella 'payloads' che ha lo scopo di testare la funzione.

```
# Per testare una funzione
nanofaas invoke funzione-esempio -d @payloads/happy-path.json
```

La funzione viene invocata con i payloads come input. Attualmente non è presente un metodo per confrontare automaticamente i risultati con dei valori attesi ma l'output va interpretato "manualmente".

Sviluppo in locale e fn-init Un problema di fn-init che è stato rilevato riguarda la scrittura del file 'function.yaml'.

All'interno di questo file viene scritto in automatico:

```
executionMode: DEPLOYMENT
```

Questa è l'unica modalità supportata da fn-init. Le altre modalità di esecuzione documentate dalla piattaforma (LOCAL, POOL, DEPLOYMENT) non

sono selezionabili. Per eseguirle si è costretti a cambiare il file a mano.

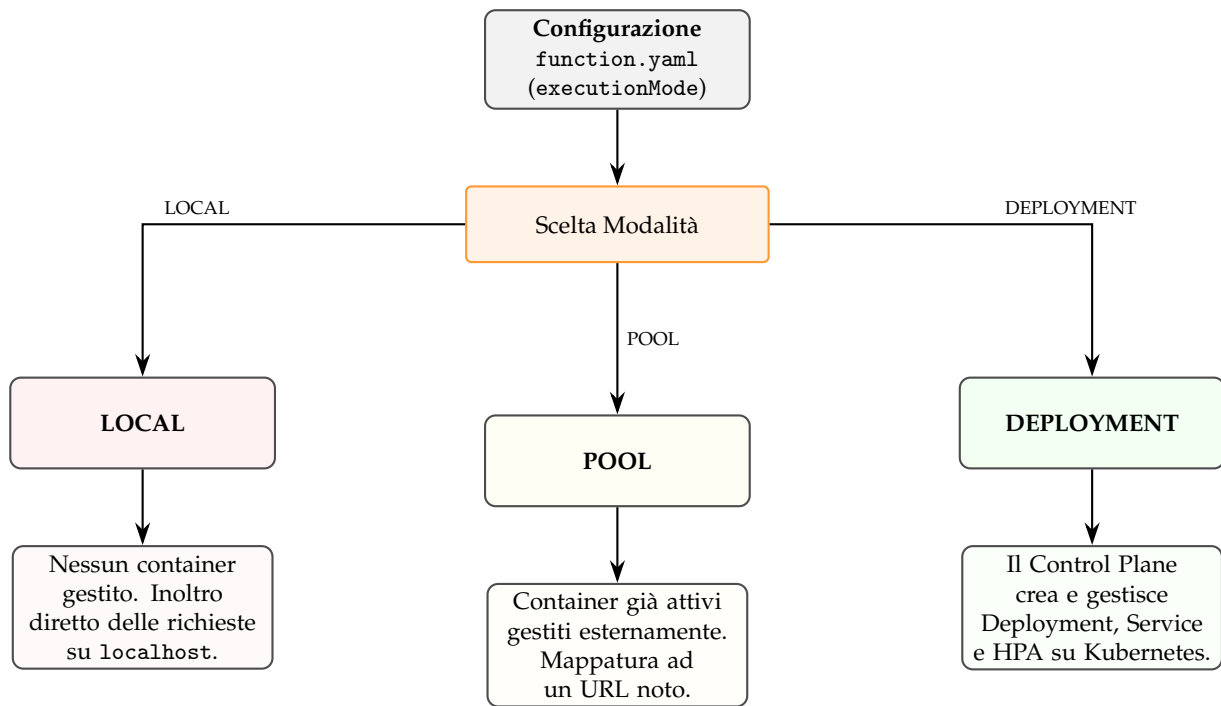


Figura 5.3: Relazione e comportamento delle modalità di esecuzione (executionMode) in nanofaas

Endpoint Locale Lo sviluppo in modalità LOCAL quindi presuppone che la funzione giri direttamente sulla macchina host. Per fare questo, il control plane deve sapere dove si trova la funzione (per esempio `http://localhost:8080`). Attualmente `fn-init` non prevede alcuna opzione per iniettare questa informazione essenziale all'interno dello YAML in fase di generazione. Si è nuovamente costretti ad aggiungere a mano un'altra informazione.

Capitolo 6

Automatizzare l'esplorazione

In linea con le premesse esposte nei capitoli precedenti, si è deciso di procedere con l'esplorazione di nanofaas in modo più formale. L'esplorazione della piattaforma in maniera informale è stata un problema.

I salvataggi delle sessioni del terminale infatti non sono stati soddisfacenti.

Rileggere centinaia di righe di comandi, con annessi i vari errori, non è per nulla efficiente dal punto di vista temporale.

Esplorazione, non testing In questi script è stata automatizzata l'esplorazione. Non sono script con la funzione di testare, infatti non c'è un risultato con cui comparare gli output finali. Sono presenti solamente dei semplici controlli durante l'esecuzione per evitare errori e velocizzare il lavoro. [2]

Riassumendo:

- Testare nanofaas nel suo reale ambiente di utilizzo (Kubernetes)
- La nuova funzionalità di scaffolding ha permesso di velocizzare enormemente il lavoro, ma manca il supporto per lo sviluppo in locale
- Isolare ogni esplorazione della piattaforma
- Documentare meglio il lavoro svolto

6.1 La struttura degli script

6.1.1 Justfile

Per semplificare il lavoro è stato usato justfile, che è un miglioramento di makefile.

È uno strumento che permette di eseguire comandi da terminale indipendentemente dalla posizione attuale. Permette anche di visualizzare una lista (adeguatamente commentata) che mostri e spieghi i comandi che possono essere avviati. [21] [5]

Con justfile ogni script che viene avviato è un processo figlio. Questo significa che è possibile iniettare delle variabili agli script, come il tipo di distribuzione linux che si vuole usare nei container o come chiamare lo snapshot della macchina virtuale.

Questo metodo torna utile se si vuole esplorare la piattaforma nanofaas in modi diversi. Esistono tre tipi di script che possono essere avviati:

- just setup → è uno script obbligatorio per impostare l'intero sistema, soprattutto la prima volta che lo si utilizza.
- just "script" → viene eseguito uno script specifico
- just test-all → esegue i principali script

6.1.2 Setup del sistema

L'esecuzione del comando "just setup" prepara sia la macchina virtuale sia l'ambiente nanofaas.

Isolare ogni esplorazione Al fine di ottimizzare i tempi di esecuzione, si è sfruttata la funzionalità di snapshot offerta da Multipass. Una volta creata la macchina virtuale e installato nanofaas, viene immediatamente eseguito uno snapshot. In questo modo è sufficiente ripristinare la macchina virtuale ad ogni sessione di test, evitando di ripetere la configurazione iniziale. Inoltre, nel corso delle verifiche è stato riscontrato che in determinate circostanze lo script poteva subire interruzioni. Era quindi possibile procedere nel seguente modo:

- Correggere l'errore e manualmente digitare i comandi rimanenti per impostare correttamente l'ambiente
- Correggere l'errore e poi cancellare la macchina virtuale, impostando l'ambiente da zero

Sono stati numerati anche i principali passaggi del processo di setup per riprendere l'esecuzione dello script, in caso si verificasse un errore. [5]

Se `START_STEP>1`

 riprendi dallo step X e alla fine crea lo snapshot

Se `START_STEP=1` e la VM esiste con snapshot

 esegui "multipass restore"

Se `START_STEP=1` ma la VM non esiste

 esegui "setup-nanofaas.sh", crea la VM e lo snapshot

Altrimenti

 Lo snapshot è mancante, creo VM e snapshot

Registry Locale Per eliminare la dipendenza da registry esterni, lo script avvia un registry container locale (registry:2) sulla porta 5000 della VM e configura k3s per usarlo come mirror. Le immagini del Control Plane vengono costruite con Docker e caricate direttamente su questo registry. Al deploy, k3s le recupera localmente tramite containerd, senza mai uscire dalla VM.

Funzioni condivise Una delle problematiche più ricorrenti riscontrate durante l'esplorazione di nanofaas ha riguardato il tempo di attesa necessario per l'avvio o il riavvio di alcuni componenti. Kubernetes espone `kubectl rollout status` per attendere il completamento di un Deployment, ma questo non garantisce che il Pod sia effettivamente in grado di rispondere. Il rollout può risultare completato mentre il container è ancora in fase di inizializzazione del runtime oppure di caricamento delle dipendenze. [31]

A tale scopo è stata implementata la funzione `wait_for_Pod_ready`, la quale attende il completamento del rollout di Kubernetes (con un timeout di due minuti) e successivamente interroga il Pod per verificarne l'effettiva disponibilità.

Solo quando entrambe le condizioni sono soddisfatte, si può dire che il Pod esiste ed è in grado di rispondere.

La funzione `deploy_and_test` risolve invece il problema di dover ripetere manualmente la stessa sequenza di operazioni per ogni funzione: costruisce l'immagine con `docker build` (gestendo automaticamente i casi particolari dei vari SDK), la carica sul registry locale, aggiorna il manifest YAML, registra la funzione e infine la invoca per verificarne il corretto funzionamento, "riassumendo" l'intero flusso in una singola chiamata. [5]

6.2 Script

6.2.1 Test API

È stata testata la capacità di nanofaas di comunicare con servizi esterni. Sono stati testati i quattro principali linguaggi: Python, Java, Go e GraalVM. [5]

Lo pseudo codice delle quattro funzioni è il seguente:

```
funzione handle(richiesta):
    try:
        risposta = chiamata HTTP ad una API_ESTERNA
        return risposta
    catch Eccezione:
        return {"error": eccezione}
```

Questa prima esplorazione ha voluto valutare fin subito la capacità di nanofaas di eseguire delle funzioni semplici e di poter comunicare con servizi web.

In tutti i casi la funzione è stata registrata, invocata ed eseguita con successo. GraalVM, ovviamente, ha richiesto spesso una compilazione molto lunga (minimo 20 minuti).

6.2.2 Multi Stage in Go

Si è provato a compilare e impacchettare una funzione Go, ma questa volta invece di installare Go sulla macchina virtuale e compilare lì, si lascia eseguire tutto a Docker.

Due stadi Un Dockerfile normale costruisce l'immagine in un unico passaggio → installazione, compilazione e impacchettamento. Il problema è che l'immagine finisce per contenere anche il compilatore, i file temporanei, la cache di apt e altre cose che servono solo per costruire, non per eseguire. Qui invece si fa così:

1. **Primo stadio:** si parte da `golang:1.25-alpine`, che è un'immagine con Go già installato e basta l'essenziale. Dentro a questo contenitore temporaneo si copia il codice della funzione e si compila tutto in un unico file binario. La compilazione è statica, il che significa che il binario non ha bisogno di librerie condivise per funzionare. È tutto dentro.
2. **Secondo stadio:** si parte da `scratch`. Scratch non è una distribuzione Linux, non ha niente. Dentro si mettono solo due cose: il binario compilato e i certificati delle autorità di certificazione (CA). I certificati servono perché la funzione deve fare chiamate HTTPS verso l'esterno e senza quelli non può verificare la firma dei server con cui parla.

Il risultato è piccolo. Circa 2-5 MB per tutta l'immagine. Niente shell, niente package manager, niente `ls`, niente `bash`.

Come arriva l'SDK dentro al container Attraverso i **contesti nominati** (`-build-context`) di Docker è possibile specificare una cartella locale (in questo caso l'SDK Go) e usarla nel Dockerfile, senza includerla nel contesto di build principale.

I file dell'SDK non vengono impacchettati e inviati al demone Docker insieme al codice della funzione, ma montati direttamente, risparmiando tempo e banda. [5] Il resto del processo di build rimane invariato.

Un approccio serverless Questo approccio è stato testato per verificare la predisposizione al serverless → la funzione è un oggetto atomico e senza dipendenze dall'ambiente che la ospita. [13]

Tutte le altre funzioni provate durante l'esplorazione sono state avviate con una `build single-stage`, per questioni di semplicità.

6.2.3 Test Comunicazione Asincrona

In `nanofaas`, l'invocazione asincrona di una funzione si differenzia da quella sincrona per il meccanismo di accodamento. Aniché invocare direttamente l'endpoint della funzione, il client chiama l'endpoint `:enqueue`, che inserisce la richiesta in una coda interna gestita dal control plane. Il flusso è il seguente:

1. Il gateway riceve la richiesta e risponde immediatamente con `202 Accepted`, restituendo un `executionId` univoco e uno stato `"queued"`.
2. Lo scheduler interno preleva il task dalla coda e lo invia tramite HTTP POST al Pod della funzione, che lo esegue.
3. Al completamento, l'esito della funzione viene notificato al control plane attraverso due canali:

- Il body della risposta HTTP 200 che il Pod restituisce al control plane.
- Un callback separato all'endpoint `:complete`, inviato dal Pod dopo aver risposto.

Entrambi i canali convergono nella stessa funzione `completeExecution` del control plane, che marca il record dell'esecuzione come SUCCESS o ERROR e rilascia lo slot di esecuzione in-flight. In caso di errore, il sistema può automaticamente accodare un retry con lo stesso `executionId`.

Questo meccanismo permette di disaccoppiare l'invio della richiesta dalla sua esecuzione: il client non deve attendere il completamento della funzione, ma può recuperare il risultato in un secondo momento interrogando l'endpoint `GET /v1/executions/{executionId}`. [7]

Per valutare il comportamento di nanofaas in modalità asincrona, sono stati simulati tre scenari diversi.

Invocazione Asincrona Questo test verifica il flusso asincrono end-to-end su una singola funzione Python, `async-python`. [5] L'handler è volutamente semplice: esegue `time.sleep(3)` per simulare un'operazione lenta, poi restituisce un messaggio di conferma e l'input ricevuto. Il flusso operativo è il seguente:

1. L'invocazione asincrona avviene con `nanofaas enqueue async-python -d '{"input": "test asincrono"}'`. Il control plane risponde con un JSON contenente `executionId` (UUID) e stato `"queued"`, senza attendere l'esecuzione della funzione.
2. Lo script estrae l'`executionId` dalla risposta con `grep -oP` e si mette in attesa di un input manuale (`read -r`). Questa pausa interattiva non è adatta alla CI, ma permette di osservare il disaccoppiamento tra invio ed esecuzione: la funzione sta già elaborando in background mentre l'utente può ispezionare lo stato del sistema.
3. Alla pressione di Invio, lo script avvia un polling sull'endpoint `nanofaas exec get {executionId}` con sei tentativi a intervalli di 3 secondi, finché lo stato non diventa `"success"`.

La risposta finale contiene, oltre a `status` e `output`, anche i `timestamp` di inizio e fine esecuzione e i campi `coldStart` (che indica se il Pod è stato avviato da zero) e `initDurationMs` (il tempo di inizializzazione in millisecondi), informazioni preziose per valutare la latenza di cold start della piattaforma.

Invocazione Asincrona Multipla - Test Ping-Pong L'obiettivo di questa esplorazione è capire se più funzioni possano collaborare utilizzando esclusivamente il control plane come orchestratore di eventi, senza alcuna interazione diretta tra i Pod.

Architettura del test Vengono avviate due funzioni Python distinte, `ping-python` e `pong-python`, ciascuna in un Pod Kubernetes separato con un singolo container runtime. [5]

Prima del test, lo script imposta la variabile d'ambiente `CALLBACK_URL` sul Deployment del control plane, utilizzando il suo `clusterIP` interno (`http://$CP_IP:8080`).

I Pod delle funzioni devono conoscere l'indirizzo del control plane per inviare le successive chiamate asincrone, ma non utilizzano i Service Kubernetes per scoprirsi a vicenda, tutta la comunicazione passa attraverso il control plane.

Meccanismo di invocazione ricorsiva Il flusso segue una catena asincrona basata sull'endpoint `:enqueue`. Lo script invoca inizialmente `ping-python` passando un payload JSON con `count=10`. Ciascuna funzione, alla ricezione del payload, controlla il valore di `count`: se è maggiore di zero, attende 0,5 secondi (per non saturare istantaneamente la coda) e poi effettua una HTTP POST asincrona verso il control plane per accodare l'altra funzione con `count-1`. La catena attesa è lineare e dovrebbe produrre esattamente undici esecuzioni prima di terminare:

```
ping 10 → pong 9 → ping 8 → pong 7 → ping 6 → pong 5 → ping
      4 → pong 3 → ping 2 → pong 1 → ping 0 (fine)
```

Quando una funzione riceve `count=0`, restituisce semplicemente il risultato senza effettuare ulteriori chiamate, interrompendo la catena.

Monitoraggio e validazione Il test campiona i log dei due Pod ogni due secondi per 30 secondi, cercando le occorrenze di `PING ACTIVE` e `PONG ACTIVE`. Allo scadere del timeout, verifica che nei log di almeno uno dei due Pod compaia un'esecuzione con `count=0`: se presente, il test è considerato riuscito, poiché significa che il countdown è arrivato a termine senza blocchi o perdite di messaggi.

Bug riscontrato - idempotenza L'esecuzione di ping-pong non ha dato problemi. Però durante l'esplorazione del Control Plane, per capire come poter testare e scrivere al meglio lo script, ci si è imbattuti in una riga interessante.

```
return new InvocationResponse(record.executionId(), "queued", null,
    null);
```

Nanofaas supporta l'idempotenza tramite una chiave (`Idempotency-Key`) inviata dal client.

Se la stessa chiave viene riutilizzata, il control plane riconosce l'esecuzione già esistente e non la crea una seconda volta.

Tuttavia, nella gestione delle funzioni asincrone è stato osservato un "queued" hardcoded nella risposta. Questo ha fatto supporre che nanofaas fosse in grado di capire se un'invocazione fosse già finita con `SUCCESS` (oppure `ERROR`). Però il "queued" hardcoded obbliga il client ad aspettare inutilmente.

6.2.4 Idempotenza

Per verificare quanto emerso durante l'esplorazione del Control Plane, è stato scritto uno script dedicato che invoca due volte, in modo asincrono, la stessa

funzione con la medesima Idempotency-Key, attendendo cinque secondi tra le due chiamate affinché la prima esecuzione possa completarsi.

Il test La prima invocazione restituisce, come atteso, uno stato `queued`. Trascorsi i cinque secondi, la seconda invocazione (stesso `executionId`, stessa chiave) restituisce anch'essa `queued`, nonostante l'interrogazione diretta dello stato dell'esecuzione tramite l'endpoint del Gateway confermi che il task fosse in realtà già concluso con esito `success`. Il test conferma quindi quanto ipotizzato analizzando il codice sorgente: la seconda richiesta si limita a riciclare l'`executionId` dell'esecuzione esistente, ignorando lo stato reale del task e rispondendo con un `"queued"` hardcoded. [5]

Causa e soluzione Il problema risiede nel metodo `invokeAsync`, che dopo aver recuperato o creato il record dell'esecuzione tramite `InvocationExecutionFactory`, restituiva sempre una risposta con stato `"queued"`, senza controllare se il record fosse già in uno stato terminale (`SUCCESS` o `ERROR`).

La soluzione proposta introduce un controllo preliminare: prima di procedere con l'accodamento, viene verificato se il record dell'esecuzione ha già raggiunto uno stato terminale. In caso affermativo, viene restituita immediatamente la risposta con il risultato reale, senza generare una nuova voce in coda; in caso contrario, il flusso prosegue come in precedenza, accodando l'esecuzione e restituendo lo stato `"queued"`. Questo distingue correttamente il caso di *replay* (una richiesta con chiave già nota e completata) da quello di una nuova invocazione, senza intaccare il meccanismo di deduplica già gestito da `InvocationExecutionFactory` e da `InvocationEnqueueSupport`, che restano responsabili di evitare il riaccodamento di un'esecuzione già esistente.

Applicata la correzione, il test è stato rieseguito con esito positivo: la seconda invocazione restituisce ora lo stato reale dell'esecuzione anziché il valore hardcoded, confermando che il bug è stato risolto senza introdurre regressioni nel comportamento di deduplica.

6.2.5 Dipendenze Mancanti

Questo test indaga un aspetto critico di qualsiasi piattaforma FAAS: il divario tra lo stato dell'infrastruttura e la reale capacità della funzione di eseguire il proprio codice.

Il problema L'handler della funzione `stupid-import-python` contiene deliberatamente un `import non_existent_module`, ma l'istruzione non è posta in cima al file, bensì all'interno del corpo della funzione `handle()`. [5] Questa scelta non è casuale: in Python, un `import` a livello di modulo viene eseguito al caricamento del file, impedendo al processo di avviarsi e facendo fallire immediatamente anche il readiness probe. Nascondendo l'`import` dentro la funzione, invece, il modulo viene caricato senza errori, il server HTTP si avvia correttamente e l'endpoint `/health` risponde 200 OK. Kubernetes considera il Pod sano e pronto a ricevere traffico.

Solo al momento della prima invocazione della funzione, l'import fallisce con `ModuleNotFoundError`. Il test mette quindi in luce un *false positive* del readiness probe: un Pod perfettamente funzionante dal punto di vista dell'infrastruttura (processo attivo, porta in ascolto, handshake TCP riuscito) è del tutto incapace di svolgere il proprio compito applicativo.

Cold start e warm start Il test effettua due invocazioni consecutive sulla stessa istanza del Pod.

La prima è un cold start. Il Pod è appena stato creato e non ha ancora processato alcuna richiesta.

La seconda è un warm start. Il Pod ha già servito la prima invocazione e il runtime è caldo. L'obiettivo è verificare che l'errore sia *deterministico e coerente* in entrambi i casi:

- Nella prima invocazione il test si aspetta un errore 500 con `ModuleNotFoundError`, e lo verifica cercando la firma dell'errore nel body della risposta.
- Nella seconda invocazione il test si aspetta *esattamente lo stesso errore*. Se il comportamento cambiasse (ad esempio, se il Pod andasse in crash dopo il primo errore, o se una qualche forma di caching mascherasse il fallimento), il test fallirebbe.

Il "test" cerca di verificare che

- **Assenza di controlli pre-deploy:** nanofaas non esegue alcuna forma di analisi statica o linting del codice prima del deploy. La validazione delle dipendenze è interamente delegata allo sviluppatore e alla correttezza del Dockerfile. Se un `pip install` omette una dipendenza, la piattaforma non se ne accorge finché la funzione non viene invocata.
- **Superficialità del readiness probe:** Il probe predefinito (`/health`) verifica solo che il server HTTP sia in ascolto, non che il codice applicativo sia eseguibile. La distinzione tra *infrastructure readiness* e *application readiness* non è colmabile senza un probe specifico per l'applicazione. [31]
- **Robustezza del runtime:** L'errore non causa il crash del Pod. Il runtime Python e il server uvicorn rimangono attivi e continuano ad accettare richieste, restituendo l'errore a ogni invocazione. Questo comportamento è corretto, un errore applicativo non deve mandare giù l'infrastruttura e la piattaforma gestisce il fallimento in maniera controllata.
- **Coerenza cold/warm:** Il comportamento degli errori è identico tra cold e warm start. Non ci sono meccanismi di caching interno che, dopo il primo errore, mascherino o modifichino la risposta nelle invocazioni successive. L'output è deterministico e riproducibile.

L'esplorazione ha dimostrato che una funzione nanofaas, se scritta in maniera errata, fallisce in maniera coerente e senza danni collaterali al Pod o alla piattaforma.

6.2.6 Callback Loop

e2e-callback-loop-asserted.sh

[5] Il test esegue una funzione che ogni volta che viene eseguita, ne lancia un'altra copia di sé stessa in modo asincrono, chiamando direttamente l'API del gateway. È il peggior tipo di carico per un sistema **FAAS**: un workload che si autoalimenta, in cui ogni esecuzione genera almeno un'altra esecuzione, senza che la piattaforma abbia un meccanismo naturale di "arresto".

L'obiettivo non è verificare che il loop si interrompa, ma accertarsi che il sistema mantenga l'integrità.

L'enqueue iniziale deve essere ricevuto, la prima esecuzione deve essere completata e il *Control Plane* non deve crollare.

Come lo simula

Dentro l'`handler.py` si esegue un log, viene effettuata una chiamata POST al gateway della piattaforma per mettersi in coda nuovamente. La POST è sincrona, ma l'esecuzione successiva nasce come entità asincrona separata che il *Control Plane* dovrà schedare.

Per evitare di saturare in maniera incontrollata, il *manifest* della funzione fissa `concurrency: 10`, dunque il *Pod* può ospitare al massimo dieci esecuzioni in parallelo. Il ritmo resta sostenuto, mantenendo alta la pressione sulla coda interna.

L'esplorazione automatizzata procede in questo modo:

- **1. Chiamata iniziale:** Verifica che la CLI restituisca un *executionId*.
- **2. Completamento prima esecuzione:** Verifica che lo stato passi a *success* entro 40 secondi.
- **3. Propagazione del loop:** Controlla che dopo 6 secondi esistano log *LOOP ACTIVE*, confermando che la ricorsione asincrona funzioni.
- **4. Integrità Control Plane:** Interroga `/actuator/health/readiness` per 45 secondi. Fallisce se il sistema risulta non pronto per più di tre volte.
- **5. Fallimento chiamate:** Scansiona i log cercando *Loop error*. Se presente, indica che la piattaforma ha rifiutato richieste di re-entrata.

Considerazioni Il test attuale esercita solo il meccanismo per cui una funzione ne lancia un'altra.

Non verifica che il risultato di un'esecuzione viene memorizzato e consumato da un'altra funzione.

Capitolo 7

Analisi degli SDK

Questo capitolo analizza l'esperienza di sviluppo offerta da nanofaas attraverso i suoi vari Software Development Kit (SDK).

L'obiettivo principale è verificare se l'astrazione offerta dalla piattaforma si mantenga uniforme nei vari linguaggi supportati, oppure se emergano divergenze operative dovute alle differenze intrinseche dei linguaggi stessi.

L'esplorazione ha riguardato la consistenza architetturale degli SDK di Java, Go e Python su aspetti critici come la gestione dei dati in ingresso, la propagazione del contesto di esecuzione e le modalità di inizializzazione delle funzioni.

Sebbene questa tesi cerchi di mantenere la presentazione degli argomenti con un approccio concettuale e discorsivo, verranno presentati alcuni esempi di codice essenziali per illustrare concretamente le differenze comportamentali e l'impatto sulla robustezza del sistema.

7.1 Da un'API Comune alla frammentazione

In un ecosistema *serverless* ideale, la piattaforma dovrebbe essere agnostica rispetto al linguaggio di programmazione. L'infrastruttura sottostante, come il *Control Plane* e lo *scheduler*, dovrebbe interagire con il codice utente in modo trasparente e uniforme. L'idea di base è che una funzione, sia essa scritta in Python, Java o Go, debba comportarsi allo stesso modo di fronte allo stesso stimolo esterno.

Tuttavia, analizzando a fondo gli SDK di nanofaas, si osserva che questa uniformità operativa viene spesso sacrificata in favore di un'ottimizzazione idiomantica per il singolo linguaggio. Se da un lato questo permette agli sviluppatori di usare gli strumenti e le convenzioni a loro più familiari per ogni ecosistema, dall'altro introduce comportamenti asimmetrici in scenari critici.

Questo fenomeno genera una "Leaky Abstraction" → un'astrazione in cui le complessità e le particolarità del runtime non vengono nascoste del tutto, ma trapelano e complicano lo sviluppo, obbligando l'utente a conoscere i dettagli

implementativi della piattaforma. [1]

Attualmente nanofaas sceglie l'ottimalità (sfruttando i punti di forza specifici di ogni linguaggio) a scapito dell'uniformità operativa globale.

7.2 Gestione dei dati e dei metadati

Una delle prime asimmetrie rilevate riguarda il modo in cui il payload delle richieste viene inoltrato dal *Watchdog* al runtime della funzione. [6]

Quando una richiesta HTTP arriva alla piattaforma, il *Control Plane* raccoglie sia i dati principali della chiamata (l'input vero e proprio) sia una serie di informazioni accessorie essenziali, come chiavi di sicurezza, ID utente o identificativi per il tracciamento (i metadati). Questo pacchetto completo viene solitamente incapsulato in una "busta" standardizzata.

Negli SDK per Java e Go, la piattaforma adotta un approccio rigoroso e consegna il "biglietto completo". La funzione riceve sempre un oggetto strutturato, spesso chiamato *InvocationRequest*, che permette di accedere sia all'input sia ai metadati.

Ecco un esempio astratto di come un handler in Java gestisce questa struttura:

```
@nanofaas_function
public class HelloWorldFunction implements FunctionHandler {
    @Override
    public Object handle(InvocationRequest request) {
        // Accesso simultaneo ai dati e ai metadati
        Object data = request.input();
        Map<String, String> meta = request.metadata();

        return "Ricevuto: " + data;
    }
}
```

In questo scenario, il sistema valida preventivamente la struttura della richiesta sfruttando annotazioni rigide (come *@NotNull*). Se un client invia dati malformati, la chiamata viene respinta immediatamente prima ancora di raggiungere l'handler, proteggendo il codice interno da crash inattesi.

In Python, invece, la gestione risulta più sfumata. Il runtime espone direttamente all'handler soltanto il dato di input estratto dal payload originale.

Questa scelta progettuale non implica una perdita totale delle informazioni, quanto piuttosto una loro distribuzione su canali paralleli. I *metadati di sistema*, come l'*Execution ID* necessario al tracciamento, sono infatti gestiti dal runtime Python attraverso meccanismi separati e rimangono accessibili alla funzione.

Al contrario, eventuali *metadati applicativi*, ad esempio header HTTP personalizzati, chiavi di sicurezza o token specifici del dominio, non sono direttamente disponibili nell'interfaccia dell'handler e per poterli utilizzare lo svi-

luppatore deve ricorrere a canali alternativi non documentati o a stratagemmi implementativi.

```
@nanofaas_function
def handler(req):
    # 'req' contiene unicamente il payload applicativo.
    # I metadati di sistema (es. Execution ID) sono
    # gestiti separatamente via contextvars.
    # I metadati applicativi non sono direttamente accessibili.
    print(f"Tipo ricevuto: {type(req)}")
    return req
```

In aggiunta, in Python la mancanza di una validazione rigida all'ingresso fa sì che payload strutturalmente errati vengano comunque elaborati in modo cieco, causando inevitabili crash all'interno dell'applicativo.

7.3 Tracciamento delle richieste e contesto

In una piattaforma distribuita complessa, è fondamentale poter tracciare una singola richiesta attraverso tutte le sue fasi di esecuzione, mantenendo vivo un identificativo di correlazione univoco, comunemente chiamato *Execution ID*.

Dall'esplorazione è emerso che nanofaas gestisce questa esigenza in modi radicalmente diversi a seconda dell'SDK, riflettendo le convenzioni dei rispettivi linguaggi.

7.3.1 L'approccio Esplicito di Go

Go abbraccia una filosofia totalmente esplicita. Non esistendo un concetto di "contesto globale" nel design del linguaggio, lo sviluppatore è obbligato a propagare manualmente un oggetto di tipo `context.Context` in ogni singola funzione interna della catena di chiamate. [20]

```
func Handler(ctx context.Context, event Event) error {
    // Il contesto viene passato dal runtime e contiene Execution ID
    execId := nanofaas.ExecutionIDFromContext(ctx)

    // Lo sviluppatore DEVE propagare esplicitamente ctx
    return eseguiOperazioneInterna(ctx, event)
}

func eseguiOperazioneInterna(ctx context.Context, e Event) error {
    // se ctx non fosse stato passato la tracciabilità sarebbe persa
    return nil
}
```

Questo approccio, pur introducendo un certo grado di codice ripetitivo (*boilerplate*), garantisce una tracciabilità palese e robusta. Se ci si dimentica di passare il contesto, il compilatore non segnala errore, ma la rottura della catena di logging diventa immediatamente rintracciabile nel codice sorgente.

7.3.2 L'approccio Implicito di Java

Java delega la gestione del contesto a strumenti impliciti, in particolare al meccanismo del `ThreadLocal` (usato ad esempio dall'MDC, *Mapped Diagnostic Context*, di librerie come SLF4J). [28] In questo modo, il contesto di esecuzione viene "attaccato" al thread corrente in modo invisibile e lo sviluppatore può recuperarlo ovunque all'interno della medesima esecuzione senza doverlo passare come parametro.

```
// Accessibile da qualsiasi punto dello stesso thread senza parametri
String execId = FunctionContext.getExecutionId();
```

I limiti di questo approccio emergono con la programmazione concorrente. Se la funzione Java delega del lavoro a un nuovo thread creato con `new Thread()`, il contesto `ThreadLocal` standard non viene ereditato automaticamente.

Java offre tuttavia la classe `InheritableThreadLocal`, che consente la propagazione automatica dei valori dal thread padre al thread figlio al momento della creazione di quest'ultimo. [28]

La vera criticità si manifesta però negli scenari moderni, dove i thread non vengono creati da zero ma sono gestiti tramite **thread pool** (`ExecutorService`). In questo contesto, i thread vengono riutilizzati attraverso esecuzioni successive e né `ThreadLocal` né `InheritableThreadLocal` garantiscono un isolamento corretto.

Un thread riciclato potrebbe mantenere il contesto di una richiesta precedente (contaminazione) o, al contrario, non ricevere affatto il contesto della nuova richiesta.

È per questo motivo che framework come SLF4J MDC implementano meccanismi di propagazione manuale tramite wrapper e che le versioni più recenti di Java hanno introdotto i `ScopedValues` (associati ai `Virtual Thread`) proprio per affrontare questa limitazione strutturale.

7.3.3 L'evoluzione Asincrona di Python

Python adotta una via di mezzo, risultando ottimale per le moderne chiamate asincrone. Sfrutta infatti una libreria nativa chiamata `contextvars`. Questo meccanismo permette di mantenere isolato e coerente il contesto tra le varie *coroutine* che girano in concorrenza all'interno dello stesso thread gestito dall'`event loop`. [30]

```
from nanofaas.sdk.context import get_execution_id

async def handler(event):
    exec_id = get_execution_id() # Nessun parametro esplicito

    # Il contesto viene preservato automaticamente anche dopo un'attesa
    await some_async_call()

    # L'ID rimane lo stesso, propagato nativamente
    assert exec_id == get_execution_id()
```

Ogni *Task* asincrono in Python eredita in modo trasparente una copia del contesto al momento della creazione, impedendo che le modifiche di una *coroutine* inquinino le altre. [30]

Queste divergenze confermano ancora una volta che il modo in cui una funzione propaga il proprio contesto è legato a come il linguaggio ospite gestisce thread e concorrenza.

7.4 Inizializzazione e stati di salute apparente

Un'altra differenza operativa di grande impatto emersa durante le esplorazioni riguarda le fasi di avvio del container e il caricamento del codice della funzione. In un ambiente orchestrato da Kubernetes, il modo in cui un'applicazione comunica il proprio stato di salute è di vitale importanza per garantire l'affidabilità dell'intero sistema.

7.4.1 La gestione del caricamento nei diversi SDK

Le differenze tra gli SDK emergono in modo particolarmente evidente nella fase di avvio del container e caricamento del codice della funzione.

In Go, la compilazione nativa in un unico binario fa sì che errori sintattici o di dipendenze vengano rilevati a tempo di build.

Inoltre, un *panic* non catturato durante l'inizializzazione causa la terminazione immediata del processo, il Pod entra in stato di *CrashLoopBackOff* e Kubernetes rileva istantaneamente il problema.

Anche in Java la compilazione in bytecode garantisce la rilevazione degli errori sintattici prima dell'esecuzione, ma il caricamento delle classi (*class loading*) e la riflessione avvengono comunque a runtime. Se la classe handler specificata tramite una variabile d'ambiente risulta mancante, o se il suo iniziatore

statico solleva un'eccezione (ad esempio un `NoClassDefFoundError`), l'errore si manifesta durante l'esecuzione della JVM e, se non catturato, termina anch'esso il processo.

In Python il caricamento è interamente dinamico e demandato a `importlib` a runtime. La variabile d'ambiente `HANDLER_MODULE` indica al runtime quale modulo caricare:

```
env:
- name: HANDLER_MODULE
  value: "mypackage.handler"
```

Se questa variabile è assente o errata, oppure se il modulo referenziato contiene un errore che impedisce l'importazione (ad esempio l'import di una libreria inesistente), il caricamento fallisce.

7.4.2 Il problema della salute apparente

La questione critica, tuttavia, non risiede tanto nella natura dinamica o compilata del linguaggio, quanto nel modo in cui **l'implementazione dell'SDK** gestisce gli errori di inizializzazione. Durante l'esplorazione dell'SDK Python di nanofaas, si è osservato il seguente comportamento:

```
try:
    # Tenta di caricare il modulo utente
    importlib.import_module(HANDLER_MODULE)
except Exception as e:
    # L'errore viene loggato, ma non causa il blocco del processo
    logger.error("Failed to load handler", exc_info=True)

# Il processo prosegue e il server HTTP si avvia normalmente
yield
```

Stato di salute apparente L'errore di importazione viene catturato e registrato silenziosamente nei log, ma il processo non termina. Il server HTTP (FastAPI/Uvicorn) completa con successo la propria inizializzazione e si pone in ascolto delle richieste.

È fondamentale sottolineare che questo non è un difetto intrinseco di Python come linguaggio, bensì una scelta implementativa dell'SDK. Se l'SDK Java o Go adottasse la medesima strategia, racchiudere il caricamento dell'handler in un blocco try-catch senza terminare il processo, si verificherebbe esattamente la stessa situazione di falsa *readiness*. La differenza osservata è quindi dovuta alla mancata chiamata a `sys.exit(1)` (o equivalente) nel gestore degli errori dell'SDK Python, non a una caratteristica strutturale del linguaggio.

Le conseguenze sono gravi. Kubernetes interroga l'endpoint di salute del Pod (`/health`) e, trovando un server HTTP reattivo, giudica il container sano e lo inserisce nel circuito di produzione. Solo quando il *Control Plane* inoltra la prima richiesta reale, il runtime cerca la funzione mai registrata e risponde con un

errore HTTP 500, trasferendo così l'onere del fallimento sull'utente finale. La correzione è concettualmente semplice, invocare `sys.exit(1)` in caso di fallita importazione per forzare il crash del container, ma la sua assenza nell'implementazione attuale dell'SDK Python rappresenta una fragilità significativa della piattaforma. [31]

7.5 Conclusioni

L'esplorazione pratica degli SDK ha evidenziato come l'uniformità operativa promessa da una piattaforma FAAS sia, in molti casi, più fragile di quanto ci si aspetterebbe. Tuttavia, è cruciale distinguere *quali* differenze siano causate dai linguaggi stessi e quali invece siano riconducibili a decisioni implementative degli SDK.

Delle quattro divergenze analizzate in questo capitolo, soltanto **una** è genuinamente ancorata alle caratteristiche del linguaggio.

La propagazione del contesto di esecuzione → Go non dispone di un meccanismo di *thread-local storage* e impone il passaggio esplicito di `context.Context` per ogni chiamata. Python offre `contextvars` ottimizzato per la programmazione asincrona. Java adotta `ThreadLocal` come soluzione idiomatica. In questo caso, la piattaforma non può offrire un'interfaccia unificata senza snaturare le convenzioni del linguaggio ospite.

Questa differenza nella propagazione del contesto deve essere *documentata con chiarezza*.

Capitolo 8

Conclusione

Sintesi dei Risultati

La suite di test e l'esplorazione qualitativa hanno confermato che il flusso di base di nanofaas funziona. Registrazione, invocazione ed esecuzione di funzioni nei quattro linguaggi testati (Python, Java, Go, GraalVM) hanno restituito un esito positivo. Sono stati però individuati due difetti concreti nel Control Plane, distinti dalle difformità rispetto alla documentazione:

- Il metodo `invokeAsync` in `InvocationService.java:113` restituisce sempre lo stato "queued", anche quando l'esecuzione è già terminata con SUCCESS o ERROR, a differenza del percorso sincrono, dove questo controllo è presente. Nel test ping-pong questo bug ha causato la riesecuzione ciclica degli stessi execution ID, interrotta soltanto dal timeout dello script di test e non da un meccanismo della piattaforma.
- Durante l'esecuzione della suite e2e ufficiale, il test `k8sRegisterInvokeAndPoll` ha causato un crash del Control Plane, con conseguente fallimento a cascata di altri due test per mancata sincronizzazione temporale.

Differenze tra gli SDK

- **SDK Go** → È stato il più coerente verso i principi della piattaforma.
- **SDK Java** → Ha problemi con la propagazione del contesto (`ThreadLocal`), se la funzione delega lavoro a un nuovo thread, il contesto non viene ereditato automaticamente e nessun errore di compilazione lo segnala.
- **SDK Python** → È il SDK con i limiti più significativi:
 - i metadati della richiesta vengono scartati dal runtime prima di raggiungere la funzione
 - manca la validazione dei payload in ingresso
 - se avviene un errore di import, il server HTTP non viene bloccato e risulta quindi "pronto" secondo il probe di readiness anche quando la funzione non è mai stata caricata.

8.1 Valutazione della Maturità di nanofaas

nanofaas è un framework leggero, pensato per scopi di ricerca e didattici e nel suo stato attuale non è pronto per un ambiente di produzione. Oltre ai due bug del Control Plane e alle difformità tra SDK descritti sopra, pesano su questa valutazione:

- la scarsità di commenti nel codice sommata ad una scarsa documentazione, ha richiesto l'utilizzo di intelligenza artificiale per ricostruire l'architettura dei moduli
- lo stato dello strumento di scaffolding (`fn-init`), definito in questa stessa tesi come embrionale, genera un solo `executionMode` e non valida automaticamente l'output delle funzioni.

In conclusione Nessuno di questi limiti ha impedito il funzionamento delle funzionalità di base testate e la struttura del progetto risulta ottima per gli obiettivi pianificati.

Riflessione sul Percorso Formativo

Sono state acquisite competenze pratiche e teoriche in ambito Software Engineering e DevOps. Il lavoro ha richiesto il confronto diretto con:

- Il paradigma **Function-as-a-Service (FAAS)** e le logiche di runtime serverless.
- Tecnologie di containerizzazione e orchestrazione moderna, nello specifico **Docker** e **Kubernetes**.
- Metodologie **DevOps**, automazione della build (con Gradle) e testing end-to-end (E2E) in ambienti isolati (Multipass).
- La progettazione e il debug di ambienti linux e architetture cloud-native.

Bibliografia

- [1] J. Spolsky, «The Law of Leaky Abstractions,» in *Joel on Software*. Apress, 2004, pp. 197–202, ISBN: 9781430207535. DOI: 10 . 1007 / 978 - 1 - 4302 - 0753 - 5_26. indirizzo: http://dx.doi.org/10.1007/978-1-4302-0753-5_26.
- [2] E. Hendrickson, *Explore It!: Reduce Risk and Increase Confidence with Exploratory Testing*. Pragmatic Bookshelf, 2013, ISBN: 9781937785024. indirizzo: <https://pragprog.com/titles/ehxta/explore-it/>.
- [3] J. Beswick, *Operating Lambda: Performance optimization - Part 1*, 2021. indirizzo: <https://aws.amazon.com/blogs/compute/operating-lambda-performance-optimization-part-1/>.
- [4] Cloud Native Computing Foundation, *CNCF Cloud Native Definition v1.1*, 2024. indirizzo: <https://github.com/cncf/toc/blob/main/DEFINITION.md>.
- [5] L. Basanisi, *Esplorazione Automatizzata NanoFaaS - Script di setup e test E2E*, 2026. indirizzo: https://github.com/LucaBasanisi/Esplorazione_Automatizzata_Nanofaas.
- [6] miclav, *NanoFaaS - Architettura interna dei Pod delle funzioni*, 2026. indirizzo: <https://github.com/miclav/nanofaas/blob/1f7401ec2773e5c58dbacb16c0b8606b59785db8/docs/function-pod-architecture.md>.
- [7] miclav, *NanoFaaS - Control Plane Design (docs/control-plane.md)*, 2026. indirizzo: <https://github.com/miclav/nanofaas/blob/1f7401ec2773e5c58dbacb16c0b8606b59785db8/docs/control-plane.md>.
- [8] miclav, *NanoFaaS - Control Plane Modules, riga 7 (ServiceLoader SPI)*, 2026. indirizzo: <https://github.com/miclav/nanofaas/blob/1f7401ec2773e5c58dbacb16c0b8606b59785db8/docs/control-plane-modules.md#L7>.
- [9] miclav, *NanoFaaS - Observability (docs/observability.md)*, 2026. indirizzo: <https://github.com/miclav/nanofaas/blob/1f7401ec2773e5c58dbacb16c0b8606b59785db8/docs/observability.md>.
- [10] miclav, *NanoFaaS - Root build.gradle, riga 8 (plugin GraalVM native)*, 2026. indirizzo: <https://github.com/miclav/nanofaas/blob/1f7401ec2773e5c58dbacb16c0b8606b59785db8/build.gradle#L8>.
- [11] miclav, *NanoFaaS - System Architecture (docs/architecture.md)*, 2026. indirizzo: <https://github.com/miclav/nanofaas/blob/1f7401ec2773e5c58dbacb16c0b8606b59785db8/docs/architecture.md>.

- [12] miciaiv, *NanoFaaS - Watchdog Cargo.toml, righe 23-29 (profilo release size-optimized)*, 2026. indirizzo: <https://github.com/miciaiv/nanofaas/blob/1f7401ec2773e5c58dbacb16c0b8606b59785db8/watchdog/Cargo.toml#L23-L29>.
- [13] Amazon Web Services, *AWS Lambda - Serverless Compute*. indirizzo: <https://aws.amazon.com/lambda/lambda-functions/>.
- [14] Canonical Ltd., *Multipass*. indirizzo: <https://canonical.com/multipass>.
- [15] Cloud-init Project, *Cloud-init Documentation*. indirizzo: <https://docs.cloud-init.io/en/latest/>.
- [16] *Control Groups - Linux Kernel Documentation*. indirizzo: <https://docs.kernel.org/admin-guide/cgroup-v1/cgroups.html>.
- [17] Docker Inc., *Storage drivers (Docker Docs)*. indirizzo: <https://docs.docker.com/engine/storage/drivers/>.
- [18] *Function as a Service (FaaS) System Design*. indirizzo: <https://www.geeksforgeeks.org/system-design/function-as-a-service-faaS-system-design/>.
- [19] *GitHub Account Setup and Configuration*. indirizzo: <https://git-scm.com/book/it/v2/GitHub-Account-Setup-and-Configuration>.
- [20] Go Team, *Package context (Go standard library)*. indirizzo: <https://pkg.go.dev/context>.
- [21] *Just - a command runner*. indirizzo: <https://just.systems/man/en/>.
- [22] miciaiv, *NanoFaaS - Minimal, high-performance FaaS control plane*. indirizzo: <https://github.com/miciaiv/nanofaas>.
- [23] *NanoFaaS Issue 53*. indirizzo: <https://github.com/miciaiv/nanofaas/issues/53>.
- [24] *NanoFaaS Issue 56*. indirizzo: <https://github.com/miciaiv/nanofaas/issues/56>.
- [25] *NanoFaaS Issue 57*. indirizzo: <https://github.com/miciaiv/nanofaas/issues/57>.
- [26] *NanoFaaS Issue 65*. indirizzo: <https://github.com/miciaiv/nanofaas/issues/65>.
- [27] *opencontainers/runc*. indirizzo: <https://github.com/opencontainers/runc>.
- [28] Oracle, *Class ThreadLocal (Java SE 21)*. indirizzo: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/ThreadLocal.html>.
- [29] *Per Iniziare - Cos'è Git?* Indirizzo: <https://git-scm.com/book/it/v2/Per-Iniziare-Cos%E2%80%99C3%A9-Git%3F>.
- [30] Python Software Foundation, *contextvars — Context Variables (Python 3)*. indirizzo: <https://docs.python.org/3/library/contextvars.html>.
- [31] The Kubernetes Authors, *Configure Liveness, Readiness and Startup Probes*. indirizzo: <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>.

- [32] The Kubernetes Authors, *Horizontal Pod Autoscaling*. indirizzo: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>.
- [33] *tldr-pages*. indirizzo: <https://github.com/tldr-pages/tldr>.
- [34] *What is FaaS? (Red Hat)*. indirizzo: <https://www.redhat.com/it/topics/cloud-native-apps/what-is-faas>.
- [35] *What is Kubernetes? (Google Cloud)*. indirizzo: <https://cloud.google.com/learn/what-is-kubernetes>.
- [36] *What is Kubernetes? (Official Docs)*. indirizzo: <https://kubernetes.io/it/docs/concepts/overview/what-is-kubernetes/>.
- [37] *What's a Linux Container?* Indirizzo: <https://www.redhat.com/it/topics/containers/whats-a-linux-container>.